



“대한민국을 이끌어온 자랑스럽고 위대한 역사, 눈부신 고대의 새로운 백년도 지켜가겠습니다.”

KOREA UNIVERSITY

Deep Dive into MAL Based Attack Simulations

보안공학 연구실

2025.07.30.(수)

석사과정 49기 박준혁

목차

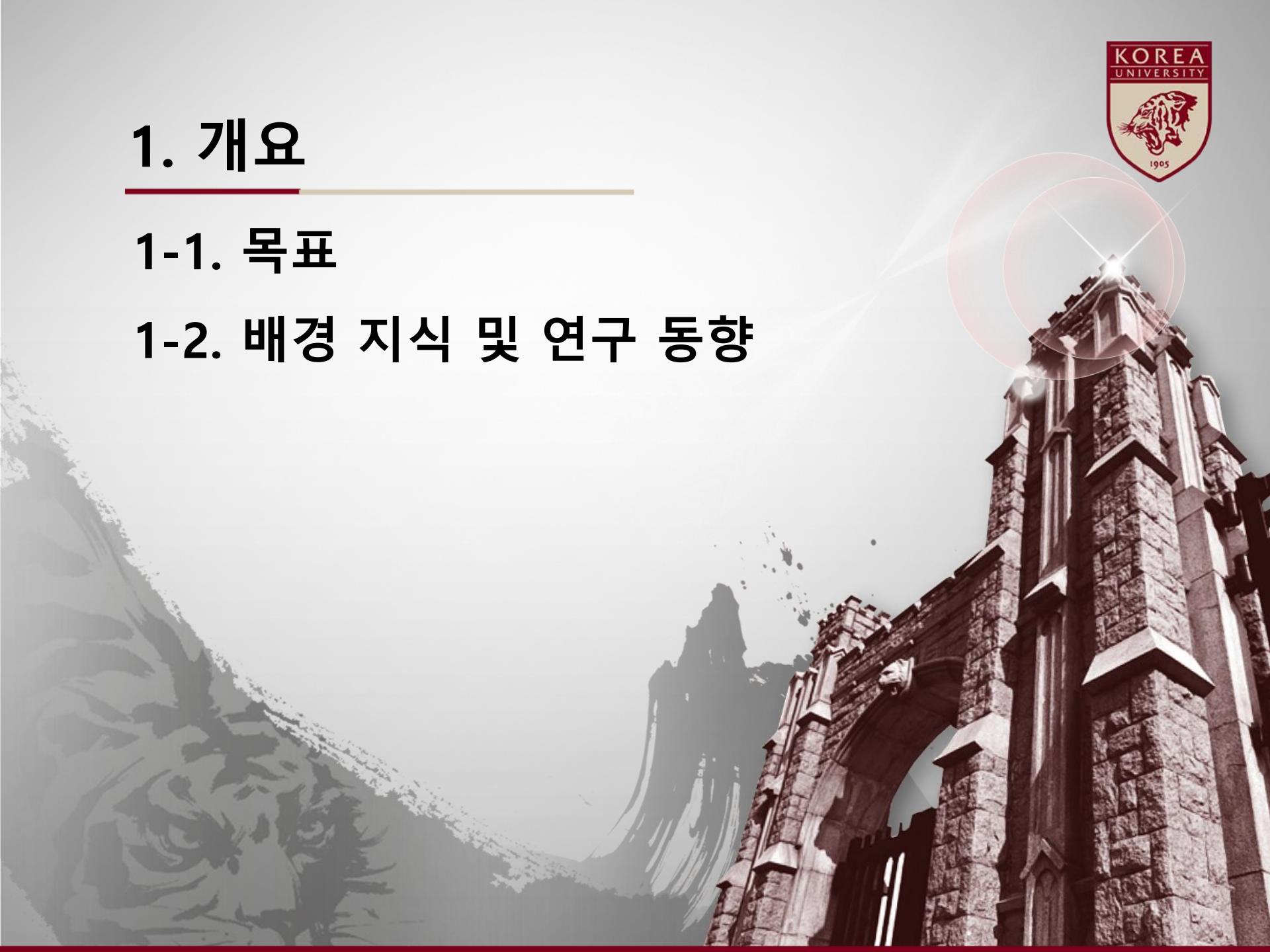
1. 개요
2. MAL을 활용한 DSL 생성
3. DSL 컴파일
4. DSL을 활용한 시스템 모델링
5. 공격 그래프 생성
6. 공격 시뮬레이션 수행
7. Wrap-Up
8. 부록



1. 개요

1-1. 목표

1-2. 배경 지식 및 연구 동향



1-1. 개요 - 목표

▪ Who

- SANE lab 구성원에게

▪ What:

- **MAL**을 이용하여 분석 대상 시스템에 대한 **공격 그래프 도출 및 공격 시뮬레이션** 수행 방법론 소개

▪ Why:

- 최근 여러 연구에서 **MAL이 위협 모델링 과정의 효율성을 높일 수 있음**을 입증함에 따라, 우리 연구실에서도 이를 핵심 연구 주제로 선정
- securiCAD와 같이 공격 그래프 도출 및 **공격 시뮬레이션 수행을 자동화**해주는 도구에 대한 접근성 제한

▪ How:

- **mal-lang github**에서 제공하는 도구 활용
- 파이썬 스크립트를 활용한 실습



■ Model-driven security engineering

- 시스템을 특정 방법에 따라 모델링
- 시스템이 준수해야 하는 보안 속성 명세
- **model checker**를 통해 시스템 모델이 보안 속성을 준수하는지 검사^[1]
- **formal verification** 가능
- 시스템을 모델링하는 “방법”, 또는 “언어” (**Domain-Specific-Language, DSL**)가 정의하는 요소들
 - 시스템의 구성 요소
 - 구성 요소 간의 관계
 - 보안 속성 (**Security Properties**)
 - 제약조건 (constraint)
 - 요구사항 (requirement)
 - 위협 (threat)
- 전통적인 **security-focused DSL**의 예시
 - UMLsec
 - SecureUML
 - SECTET



Ex) UMLsec_[2]

- 기존 UML에 stereotypes, tags, constraints 개념을 도입, 보안 속성 및 제약 조건 표현 가능

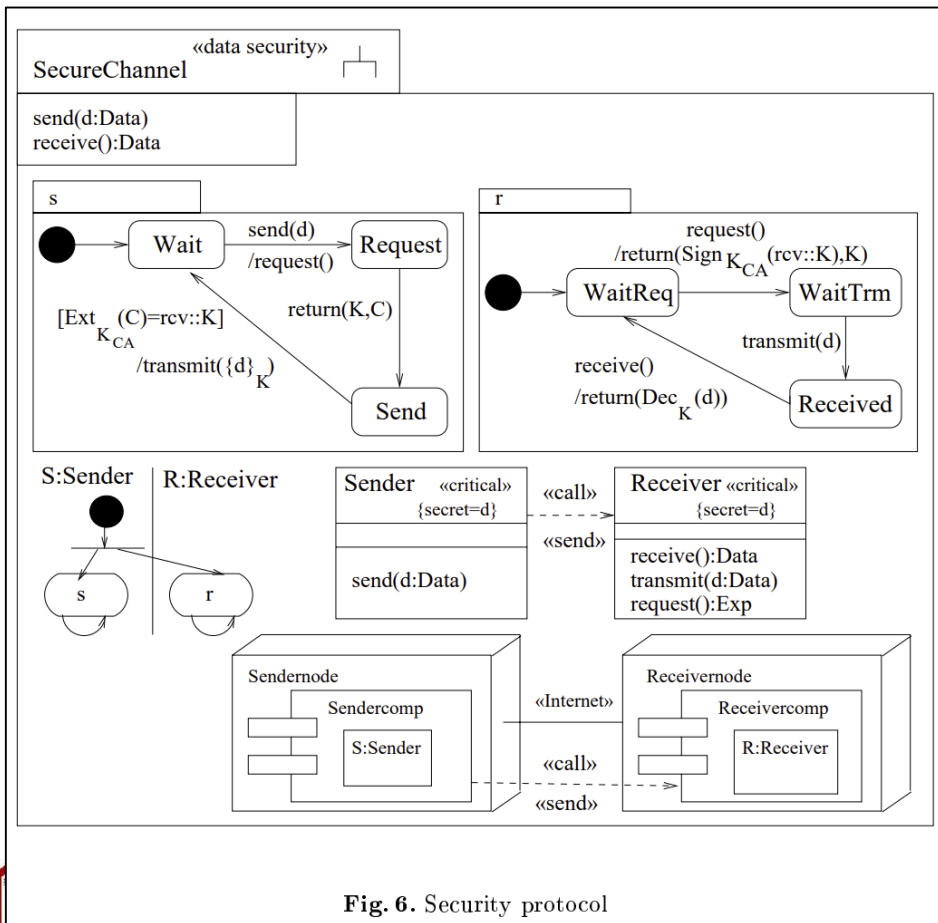


Fig. 6. Security protocol

Stereotype	Base Class	Tags	Constraints	Description
Internet	link			Internet connection
encrypted	link			encrypted connection
LAN	link			LAN connection
secure links	subsystem		dependency security matched by links	enforces secure communication links
secrecy	dependency			assumes secrecy
secure	subsystem		« call », « send » respect data security	structural interaction data security
critical	object	secret		critical object
no down-flow	subsystem		prevents down-flow	information flow
data	subsystem		provides secrecy	basic datasec
security	subsystem			requirements
fair exchange	package	start,stop	after start eventually reach stop	enforce fair exchange

Fig. 1. UMLsec stereotypes (excerpt)

■ Model-driven security engineering^[3]

- 전통적인 security-focused DSL 기반 security engineering의 한계
 - 시스템 구성 상태의 변화(다른 취약점, 다른 방어기법 등)에 둔감함
 - 특정 domain(자동차, 전력 시스템 등)에 특화된 것이 아니라 전반적인 시스템의 보안 (security)에 중점
 - model checker의 state explosion problem
 - 높은 수준의 보안 지식 요구

■ 공격 그래프(Attack Graph) & 공격 트리 (Attack Tree)

- 공격자가 수행하는 “공격 단계” 끼리 연결
- 공격에 의해 자산에 대한 해가 발생하는 과정을 보다 논리적으로 이해 가능
- 보다 효율적이고 직관적으로 시스템 구성 상태 변화에 대응 가능
- 전통적인 공격 그래프/트리의 한계
 - 공격 단계의 정의와 공격 단계 간의 연결은 시스템 모델링을 수행하는 사람의 몫
 - 높은 수준의 보안 지식 요구

■ MAL 기반 시스템 모델링 및 공격 시뮬레이션

■ MAL 기반 DSL

- “작은 **attack tree**들이 각 자산에 인코딩 되어있는” 모델링 언어

■ 장점^[3]

■ 인과관계 (causality) 기반

- 공격 단계 및 방어 기법 등 DSL 구성 요소 간 논리적 흐름 반영

■ 다양한 레벨의 **domain**에 적용 가능

- DSL 간 상속 및 연결을 지원하는 **syntax** 제공

■ 시스템 모델 및 공격 그래프의 효율적 재사용/재 구축 가능

- 시스템 모델링을 수행하는 사람은 시스템의 자산 **instance**끼리 단순 연결

■ 언어 자체의 **유지 보수성**

- 특정한 **syntax(MAL)**를 따르므로, 새로운 구성 요소 추가 및 수정이 용이

■ **고수준의 보안 지식이 필수 요건 아님**

- 시스템을 모델링 및 시스템의 보안성을 분석하기 위해 높은 수준의 보안 수준이 요구되지 않음
- 자산 **instance** 간 단순 연결을 통해 시스템 모델링 + 방어 기법 등 조건 설정



2. MAL을 활용한 DSL 생성

2-1. MAL Syntax

2-2. DSL 생성

2-3. TTC의 확률 구조



2-1. MAL을 활용한 DSL 생성 – MAL Syntax

■ MAL 구문 핵심 키워드^[4]

■ define

- 전체 MAL 프로젝트 정보 정의
- #key: "value" 형식
- #id와 #version 필수

■ include

- 다른 MAL 명세 파일 가져오기
- include "경로/파일.mal"

■ asset

- 물리적/논리적 객체를 나타내는 클래스
- Ex) 네트워크 라우터, 어플리케이션, 사용자 등

■ extends

- 상속 관계
- 부모 자산의 공격단계와 방어기법 상속

■ abstract asset

- 인스턴스화 불가능한 추상 자산
- 상속 전용

```
WITHOUT WARRANTIES O
* See the License for the spe
* limitations under the Licen
*/

#id: "org.mal-lang.coreLang"
#version: "1.0.0"

include "coreLang.mal"
```

coreLang1.0.0의 main.mal의 일부

```
include "ComputeResources.mal"
include "DataResources.mal"
include "IAM.mal"
include "User.mal"
include "Networking.mal"
include "Vulnerability.mal"
```

coreLang1.0.0의 coreLang.mal의 일부

2-1. MAL을 활용한 DSL 생성 – MAL Syntax

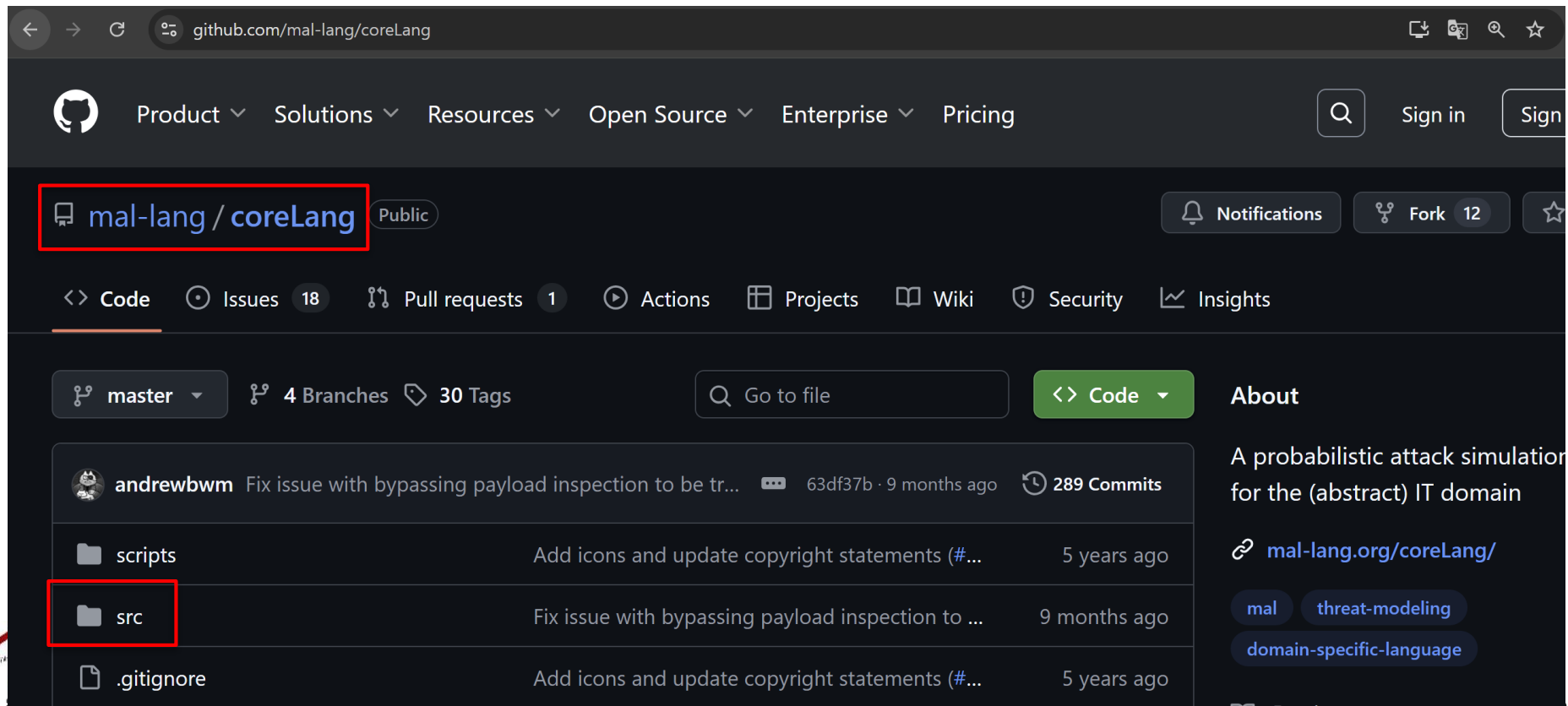
■ MAL Symbols^[4]

MAL Symbols

Symbol	Meaning	Description
->	Leads to	The successful compromise of this attack step allows the attacker to consequently compromise further attack steps. Also specifies steps affected by defenses and existence checks.
+>	Append	Child assets only. When parent and child assets have overlapping elements, e.g. <i>access</i> , the expressions defined for the child <i>access</i> are appended to those of the parent <i>access</i> . The child expressions will otherwise override those of the parent.
	OR	An OR attack step A can be reached if any of the attack steps which refers to A is reached.
&	AND	An AND attack step A can be reached only if all of the attack steps which refer to A are reached.
#	Defense	As opposed to attack steps defenses are boolean. A defense step represents the countermeasure of an attack step (e.g., passwordBruteForce attack step can be defended by twoFactorAuthentications defense step). While declaring an asset, it is possible to either enable or disable a defense step by setting the defense step to either TRUE or FALSE. It is also possible to assign a Bernoulli distribution to govern the activation.
E	Existence	It is used when the existence of connected/associated assets, given by <-, must be checked. The specified attack steps are reachable when at least one designated asset exist.
!E	Non-existence	Same as Existence, except the specified attack steps are reachable when at least one designated asset does NOT exist.
<-	Require	Denotes which associated assets are required by the current expression. Used to specify preconditions for Existence and Non-existence.
@hidden	Hidden step	(securiCAD only) Attack steps tagged as hidden remain functional but will not be shown in attack paths.

2-2. MAL을 활용한 DSL 생성 – DSL 생성

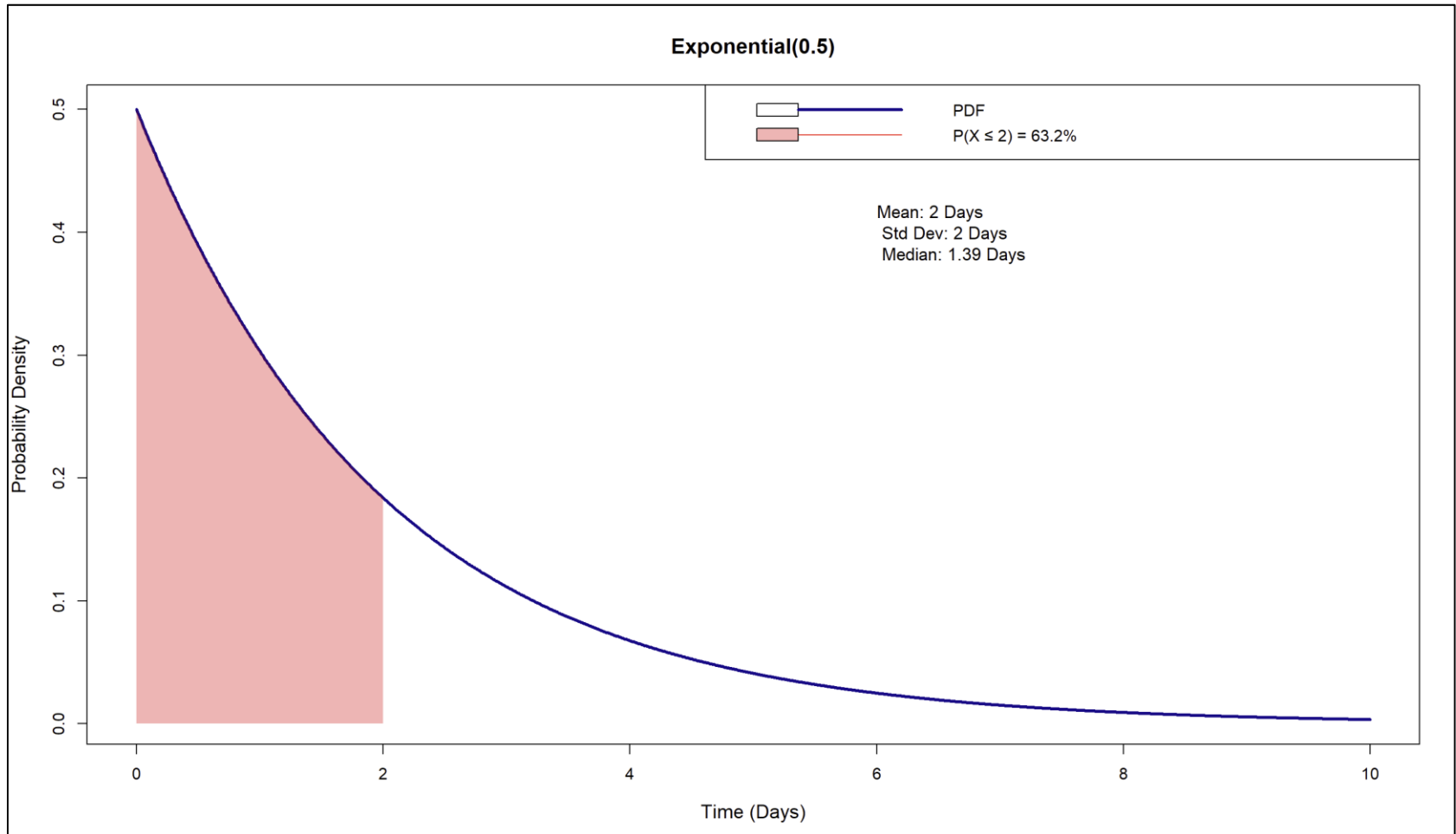
- DSL 생성 방법
 - (MAL syntax를 따라) 직접 생성
 - 공개된 DSL src 또는 컴파일된 DSL 파일 다운로드
 - Ex) coreLang 다운로드^[5]



- **TTC(Time To Compromise)에 대한 확률 구조 모델링**
 - 공격 단계를 성공시키는데 있어 걸리는 시간(**local TTC**)의 불확실성 반영
 - 각 공격 단계의 local TTC는 특정 확률 밀도에 매핑되는 **확률변수 (Random Variable)**
 - **local TTC $\sim f(t)$**
 - 특정 노드의 "local TTC는 특정 확률 밀도 함수(Probability Density Function, PDF) $f(t)$ 를 따른다"
 - **Ex) User:Phished**
 - 사용자를 **phish** 하는데 걸릴 시간이 고정인 것보다 특정 확률 밀도 함수를 따르는 것이 더욱 현실을 잘 반영함
 - **Ex) local ttc = 2일 보다 local ttc $\sim \text{Exp}(0.5)$**
 - 평균적으로 $1/0.5 = 2$ 일이 소요되지만, 분산이 $(1/0.5)^2 = 4$ 로, 어느정도 변동 가능성 있음을 의미
 - **Ex) DMZ_Net:bypassAccessControl**
 - AccessControl을 **bypass** 하는 것은 매우 어려우며, 성공 여부 자체가 불확실함
 - **local ttc $\sim \text{VeryHardAndUncertain} = \text{Bernoulli}(0.5) * \text{Exponential}(0.01)$**
 - 성공한다면 평균적으로 $1/0.01 = 100$ 일이 소요되지만, 실패한다면 해당 공격 단계는 수행 불가능
- **TTC의 확률적 특성으로 인해 공격 경로(entry point -> target)는 비결정적(non-deterministic)**



- **TTC(Time To Compromise)에 대한 확률 구조 모델링**
 - **Ex) local TTC $\sim \text{Exp}(0.5)$**



2-3. MAL을 활용한 DSL 생성 - TTC 확률 밀도 개념

- MAL에서 지원하는 TTC 확률 밀도 함수들^[6]
 - 기본 함수 또는 기본 함수의 결합 형태

- Bernoulli(p)
- Binomial(n, p)
- Exponential(λ)
- Gamma(k, θ)
- LogNormal(μ, σ)
- Pareto(x, α)
- TruncatedNormal(μ, σ^2)
- Uniform(a, b)

The following ordinal distributions are supported:

Ordinal distribution	Definition
EasyAndCertain	Exponential(1)
EasyAndUncertain	Bernoulli(0.5)
HardAndCertain	Exponential(0.1)
HardAndUncertain	Bernoulli(0.5) * Exponential(0.1)
VeryHardAndCertain	Exponential(0.01)
VeryHardAndUncertain	Bernoulli(0.5) * Exponential(0.01)

3. DSL 컴파일

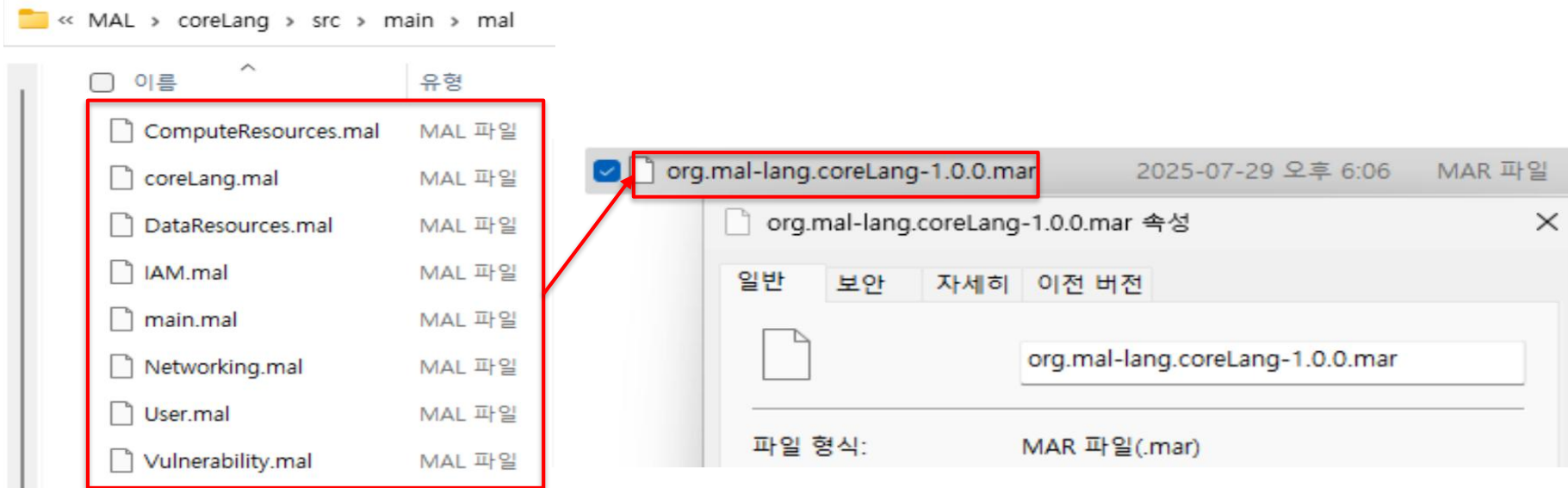
3-1. malc



3-1. DSL 컴파일 – malc

■ malc^[7]

- Mal-lang github에서 제공하는 기본 **MAL compiler**
- .mal 파일로 생성된 (여러) mal 파일들을 하나의 **DSL 파일**로 통합
- 컴파일된 **.mar** 파일은 mal_gui에서 사용 가능



■ malc 사용법

- cmd에 명령어 입력
> malc [filename]
- .mal 파일의 프로젝트 정보에 따라 DSL 파일(.mar 형식) 생성

4. DSL을 활용한 시스템 모델링

4-1. mal-gui



4-1. DSL을 활용한 시스템 모델링 – mal-gui

■ mal-gui^[8]

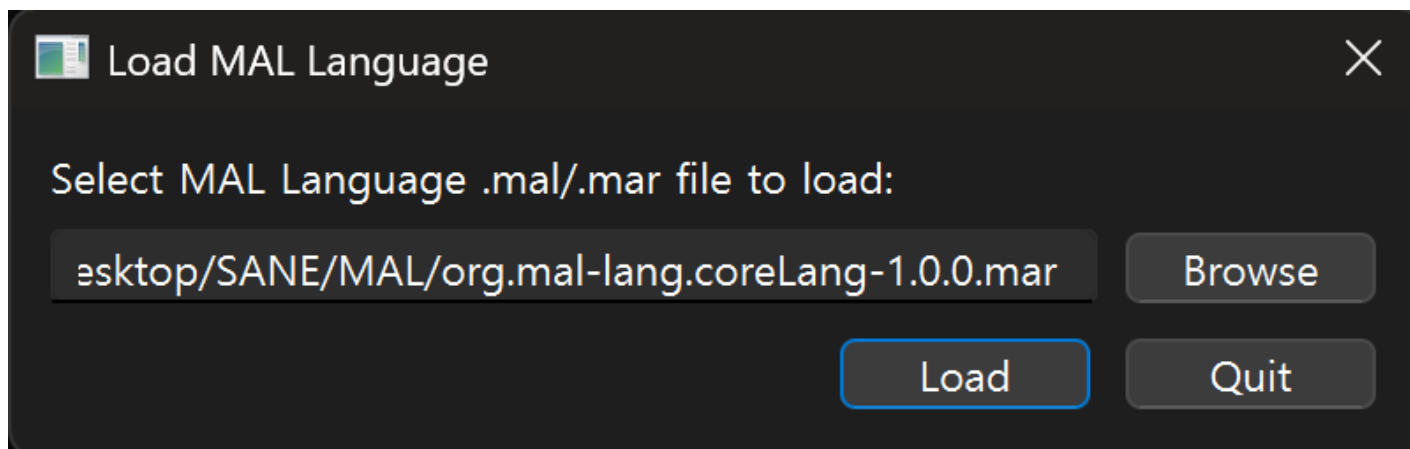
- mal-lang github에서 제공하는 **GUI 기반 시스템 모델링 도구**
- 사용자가 지정한 DSL(.mar 또는 .mal)의 규칙에 따라 시스템 모델링 가능

■ mal-gui 사용법

1. cmd에 명령어 입력

> `python -m mal_gui.app`

2. 모델링에 사용될 언어, 즉, DSL 파일 (.mar 또는 .mal) 선택



4-1. DSL을 활용한 시스템 모델링 – mal-gui

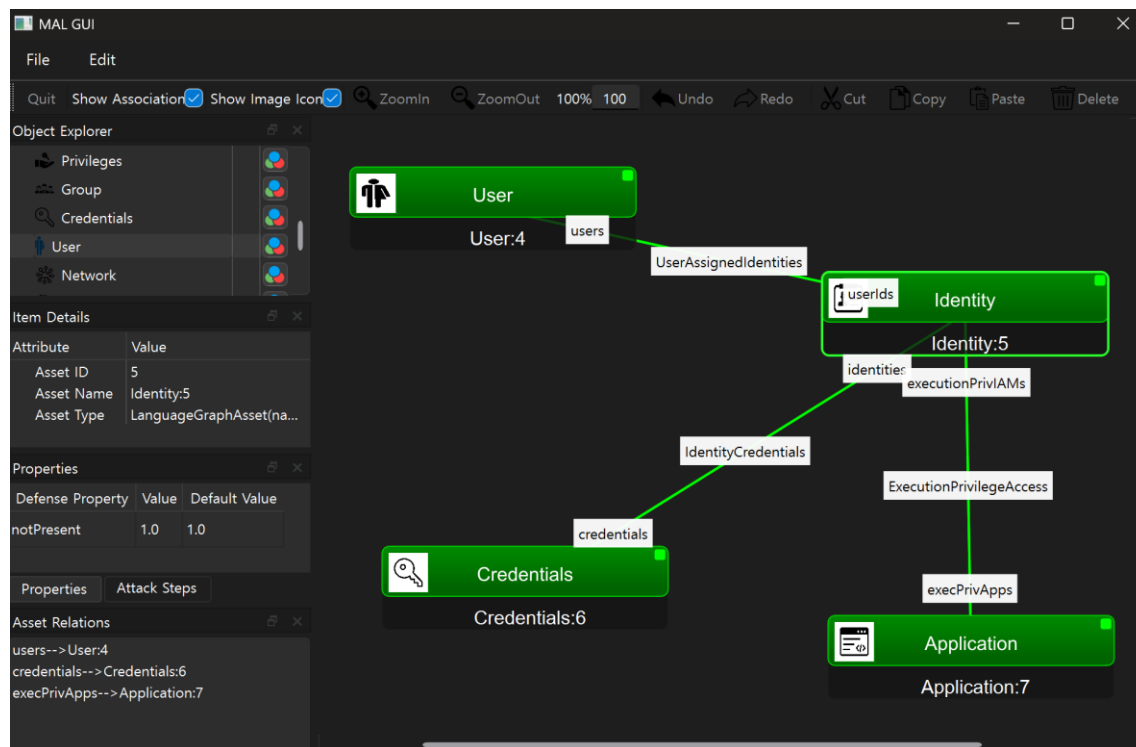
▪ mal-gui 사용법^[9]

3. Drag-and-drop 방식으로 시스템 모델링

- Object Explorer에서 자산을 drag, 작업공간에 drop
- 마우스 좌클릭 + SHIFT키를 눌러 자산과 자산 간의 association을 지정

4. 시스템 모델 (DSL instance) 불러오기 및 저장

- 좌측 상단 File을 클릭하여 시스템 모델을 yaml/json 형식으로 저장 가능

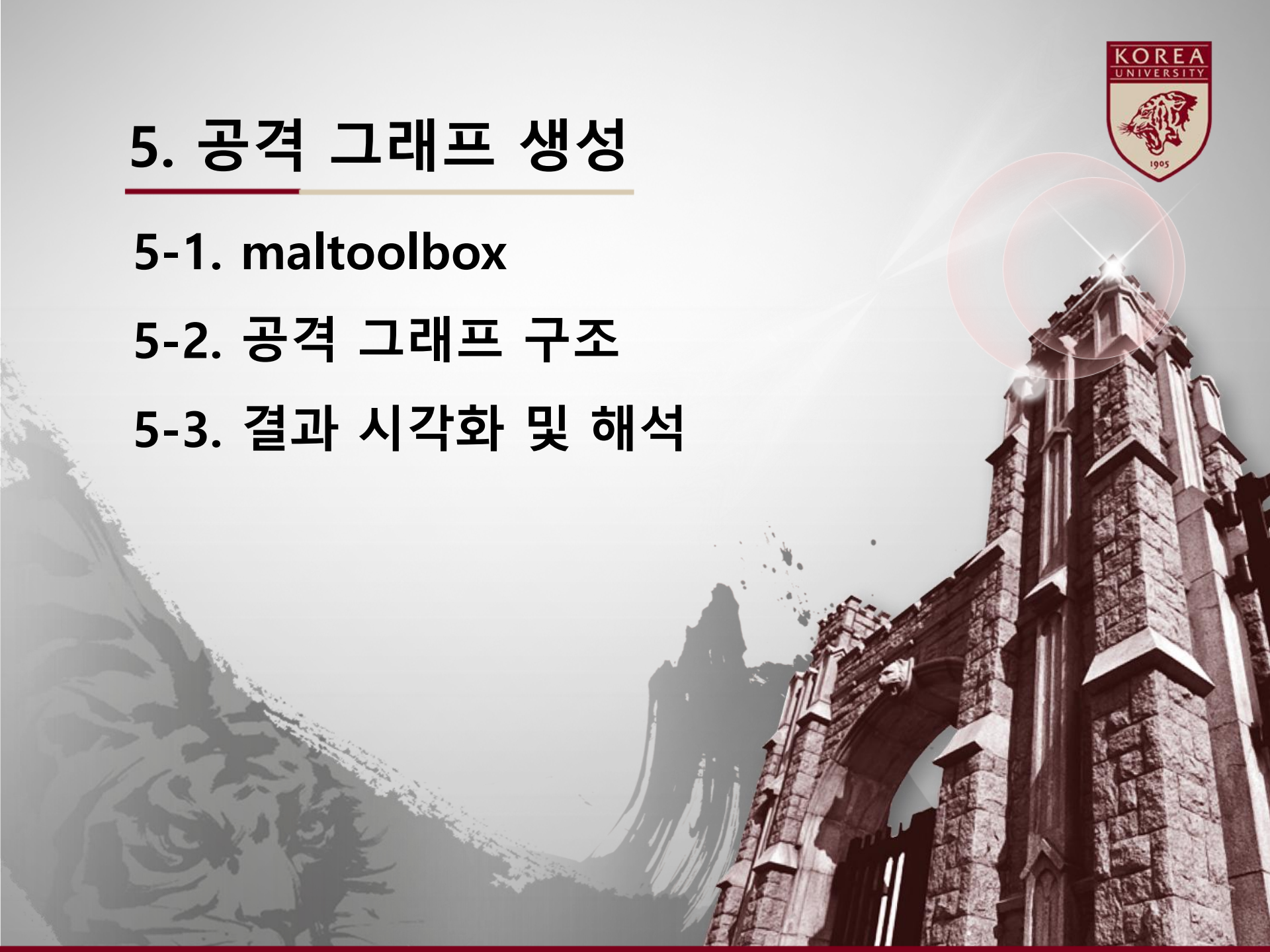


5. 공격 그래프 생성

5-1. maltoolbox

5-2. 공격 그래프 구조

5-3. 결과 시각화 및 해석



■ maltoolbox_[10]

- 공격그래프 생성 및 시각화 관련 파이썬 모듈의 집합

■ 모델링된 시스템의 전체 공격 그래프 생성

- cmd에 명령어 입력

> maltoolbox attack-graph generate [model.yml] [langFile.mar]

- 전체 공격 그래프 정보가 담긴 **attackgraph.yml** 파일 생성 (**YAML** 형식)
- **attackgraph.yml**은 작업 디렉토리의 **logs** 디렉토리에 저장

■ DSL 파일에 담긴 정보를 활용

- 공격 단계 간의 연결 관계
- 자산 간의 **association**

■ 모델링된 시스템에서 발생할 수 있는 모든 공격 단계와 방어 기법 등 구성 요소를 그래프 형식으로 연결

- 전체 공격 그래프는 공격 시뮬레이션의 기반이 됨



■ 생성된 공격 그래프(attackgraph.yml)의 구조

```
attack_steps:  
  AV:7:authCompromise:  
    asset: AV:7  
    children:  
      48: AV:7:compromise  
    compromised_by: []  
    id: 51  
    is_necessary: 'True'  
    is_viable: 'True'  
    lang_graph_attack_step: AntiVirus:authCompromise  
    name: authCompromise  
    parents:  
      49: AV:7:connect  
      50: AV:7:authenticate  
    ttc: null  
    type: and  
  AV:7:authenticate:  
    asset: AV:7  
    children:  
      51: AV:7:authCompromise  
    compromised_by: []  
    id: 50  
    is_necessary: 'True'  
    is_viable: 'True'  
    lang_graph_attack_step: AntiVirus:authenticate  
    name: authenticate  
    parents: {}  
    ttc: null  
    type: or  
  AV:7:compromise:  
    asset: AV:7  
    children:  
      33: WellProtected_OS:connect  
    compromised_by: []  
    id: 48
```

```
DB_Creds:crack:  
  asset: DB_Creds  
  children:  
    22: DB_Creds:compromise  
  compromised_by: []  
  id: 21  
  is_necessary: 'True'  
  is_viable: 'True'  
  lang_graph_attack_step: Credentials:crack  
  name: crack  
  parents:  
    19: DB_Creds:access  
  ttc:  
    arguments:  
      - 25.0  
      - 1.0  
    name: Gamma  
    type: function  
  type: and
```

노드 이름, 자산명: 공격단계

자식 노드

- 후속 공격 단계
- ...

부모 노드

- 방어 기법
- 이전 공격
- 활성화 조건
- ...

TTC 확률 밀도 함수
Ex) Gamma(25,1)

노드 유형

- 공격 단계 (and/or)

예시 attackgraph.yml의 일부, attack step 노드

예시 attackgraph.yml의 일부



5-2. 공격 그래프 생성 - 공격 그래프 구조

- 생성된 공격 그래프(attackgraph.yml)의 구조
 - 다양한 노드 type

```
DB_Creds:encrypted:
  asset: DB_Creds
  children:
    20: DB_Creds:compromiseUnencrypted
  compromised_by: []
  defense_status: '0.0'
  id: 23
  is_necessary: 'False'
  is_viable: 'True'
  lang_graph_attack_step: Credentials:encrypted
  name: encrypted
  parents: {}
  ttc: null
  type: defense
```

방어하는 공격 단계

활성화 상태
• 확률 p 로 방어

예시 attackgraph.yml의 일부, defense step 노드

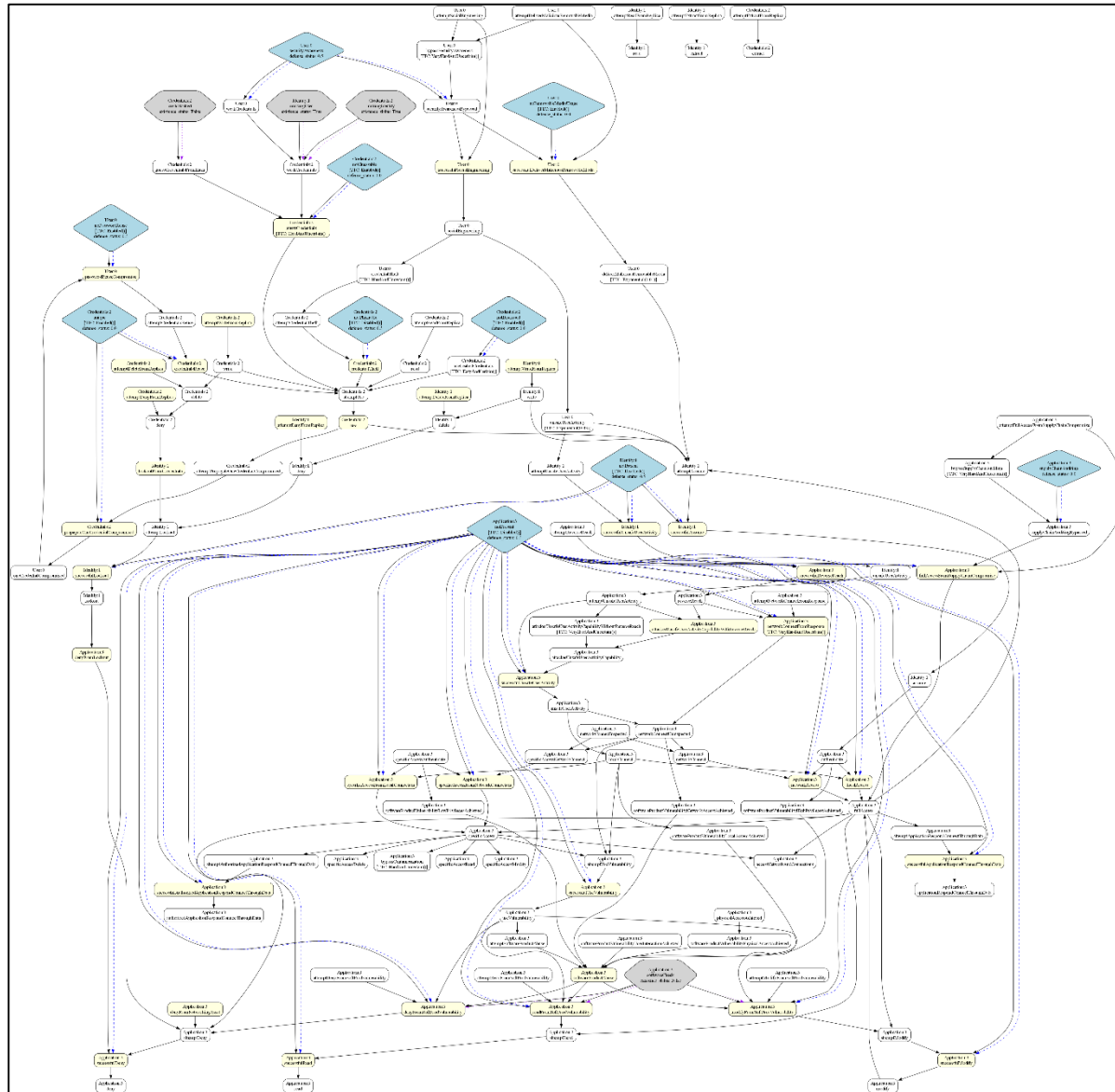
```
NotPatched_OS:antiVirusProtected:
  asset: NotPatched_OS
  children:
    7: NotPatched_OS:bypassAVwithZeroDayExploit
    8: NotPatched_OS:bypassAVwithKnownVulnExploit
  compromised_by: []
  existence_status: 'True'
  id: 14
  is_necessary: 'False'
  is_viable: 'True'
  lang_graph_attack_step: OperatingSystem:antiVirusProtected
  name: antiVirusProtected
  parents: {}
  ttc: null
  type: exist
```

조건을 영향을 받는 공격 단계

활성화 조건을 만족함

예시 attackgraph.yml의 일부, exist condition 노드

5-3. 공격 그래프 생성 - 결과 시각화 및 해석



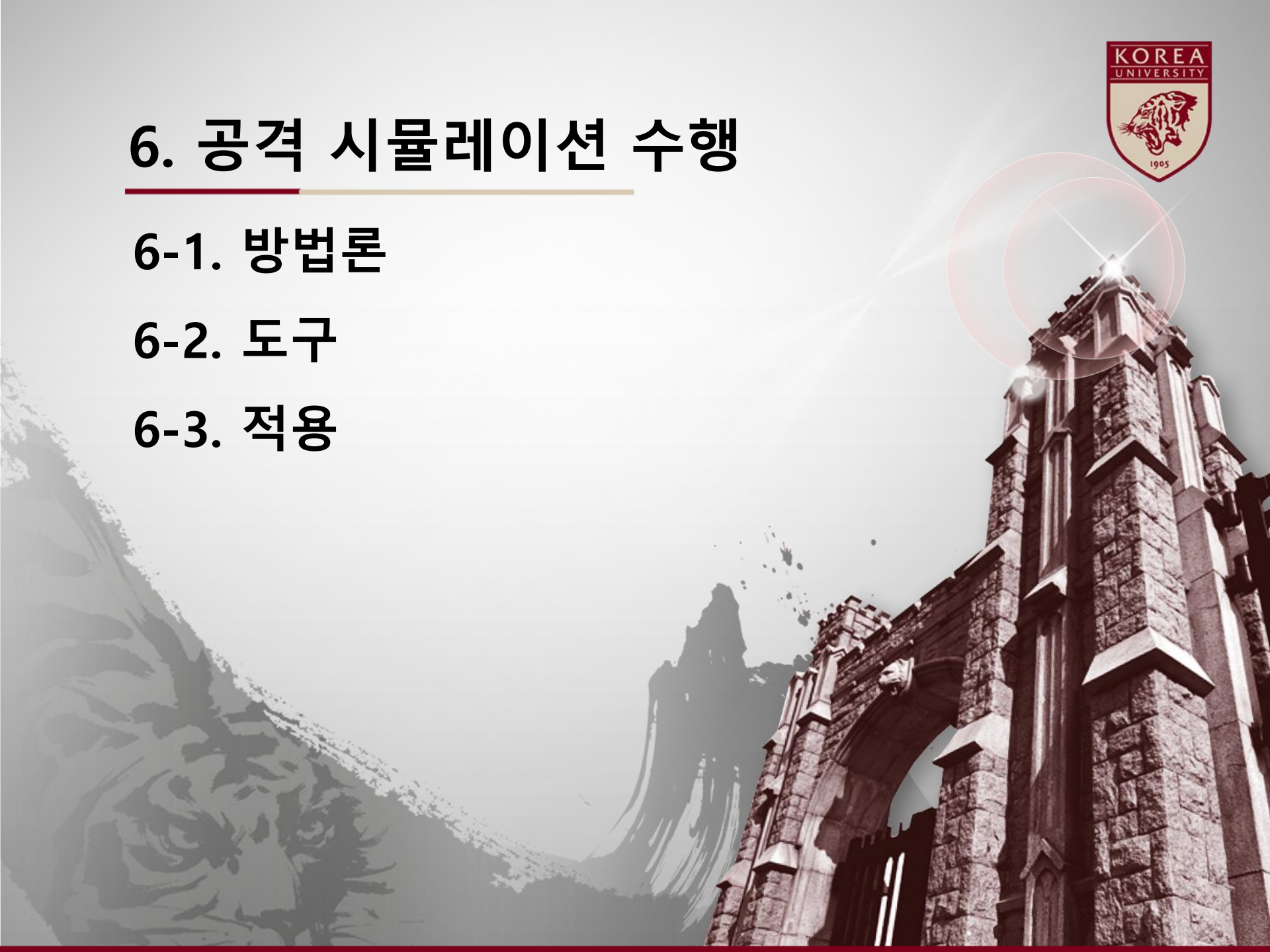
Phishing 시나리오 공격 그래프 시각화
view_attack_graph (hidden = 0) 사용(하)

6. 공격 시뮬레이션 수행

6-1. 방법론

6-2. 도구

6-3. 적용



■ 몬테 카를로 방법

- 반복된 무작위 추출(random sampling)을 이용하여 추론하고자 하는 함수 또는 특정 함수값을 수리적으로 근사하는 알고리즘
- 큰 반복 횟수(대수의 법칙)을 이용
 - 추론하고자 하는 대상이 이론적으로 복잡할 경우
- Ex) 주사위를 3번 굴려, 6이 나올 횟수 x 의 확률 밀도 함수
 - 이론적 접근법

	$\frac{5}{6} \times \frac{5}{6} \times \frac{5}{6}$ $\cong 0.5787$	$\frac{1}{6} \times \frac{5}{6} \times \frac{5}{6}$ $\cong 0.1157$	$\frac{5}{6} \times \frac{1}{6} \times \frac{5}{6}$ $\cong 0.1157$	$\frac{5}{6} \times \frac{5}{6} \times \frac{1}{6}$ $\cong 0.1157$	$\frac{1}{6} \times \frac{1}{6} \times \frac{5}{6}$ $\cong 0.0231$	$\frac{1}{6} \times \frac{5}{6} \times \frac{1}{6}$ $\cong 0.0231$	$\frac{5}{6} \times \frac{1}{6} \times \frac{1}{6}$ $\cong 0.0231$	$\frac{1}{6} \times \frac{1}{6} \times \frac{1}{6}$ $\cong 0.0046$
x	0	1	1	1	2	2	2	3
1회	x	o	x	x	o	o	x	o
2회	x	x	o	x	o	x	o	o
3회	x	x	x	o	x	o	o	o

$$\blacksquare P(X = x) = f(x) = \begin{cases} 0.5787 & (x = 0) \\ 0.3471 & (x = 1) \\ 0.0693 & (x = 2) \\ 0.0046 & (x = 3) \end{cases}$$

$$\blacksquare X \sim B\left(3, \frac{1}{6}\right), f(x) = \binom{3}{x} \left(\frac{1}{6}\right)^x \left(\frac{5}{6}\right)^{3-x} \quad (0 \leq x \leq 3, x \in \mathbb{Z})$$

6-1. 공격 시뮬레이션 수행 - 방법론

■ 몬테 카를로 방법

- 직접 주사위를 3번씩 여러 번(iteration)에 걸쳐 굴려 보고, 6이 나오는 횟수를 회차별 기록

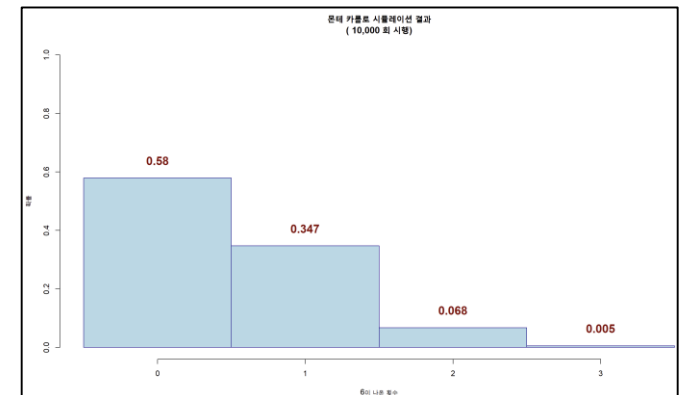
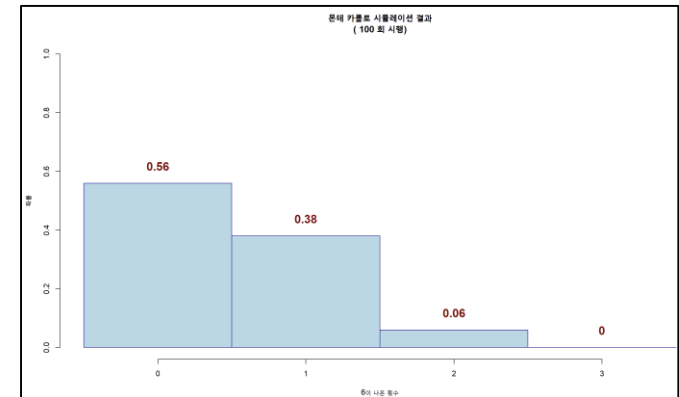
- Ex)

- 1번째에는 1회, 2번째에는 2회, 3번째에는 1회, 4번째에는 0회... 1000회까지 반복

- 6이 나오는 횟수별로 경우를 합하여 확률 추정

- Ex)

- 0번 나온 경우 : 1000회 중 578회 -> 57.8%
- 1번 나온 경우 : 1000회 중 347회 -> 34.7%
- 2번 나온 경우 : 1000회 중 69회 -> 6.9%
- 3번 나온 경우 : 1000회 중 6회 -> 0.6%



```
1 # 몬테 카를로 시뮬레이션: 주사위 3번 굴려 6이 나올 횟수
2 monte_carlo_dice <- function(iter) {
3   results <- numeric(iter)
4   for (i in 1:iter)
5   {
6     dice_rolls <- sample(1:6, 3, replace = TRUE) # 1~6 사이 숫자를 3회 비복원 추출
7     six_count <- sum(dice_rolls == 6)           # 6이 나온 횟수를 더함
8     results[i] <- six_count                     # iteration마다 나오는 결과를 저장
9   }
10  return(results)
11 }
12
13 #100회 수행
14 iter <- 100
15 result1 <- monte_carlo_dice(iter)
```

■ 몬테 카를로 방법

■ 시뮬레이션 목적 및 몬테 카를로 방법을 사용하는 이유

- 공격자가 특정 시작 단계(entry point)에서 특정 공격 단계(target)까지 도달하는데 있어 시간(Global TTC)은 어떤 패턴(Ex. 평균적으로 몇일)을 보이는가?
- Global TTC의 확률 밀도를 공격 그래프에서 이론적으로 도출하기 어려움
 - AND/OR로 복합적으로 연결된 공격 단계
 - 확률적으로 성공하는 방어 기법
 - MAL로 표현할 수 있는 다양한 조건들

■ 몬테 카를로 기반 공격 시뮬레이션 수행^[13]

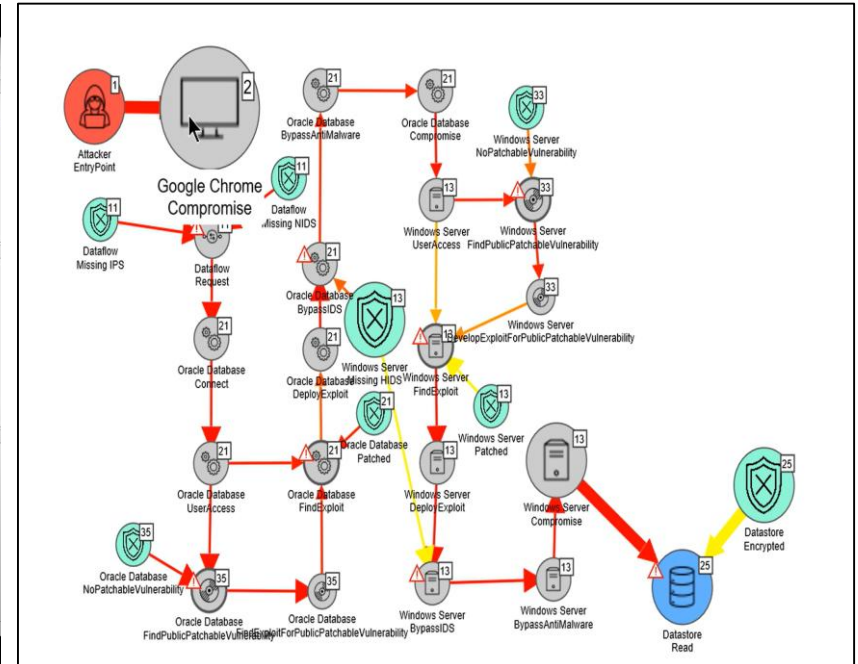
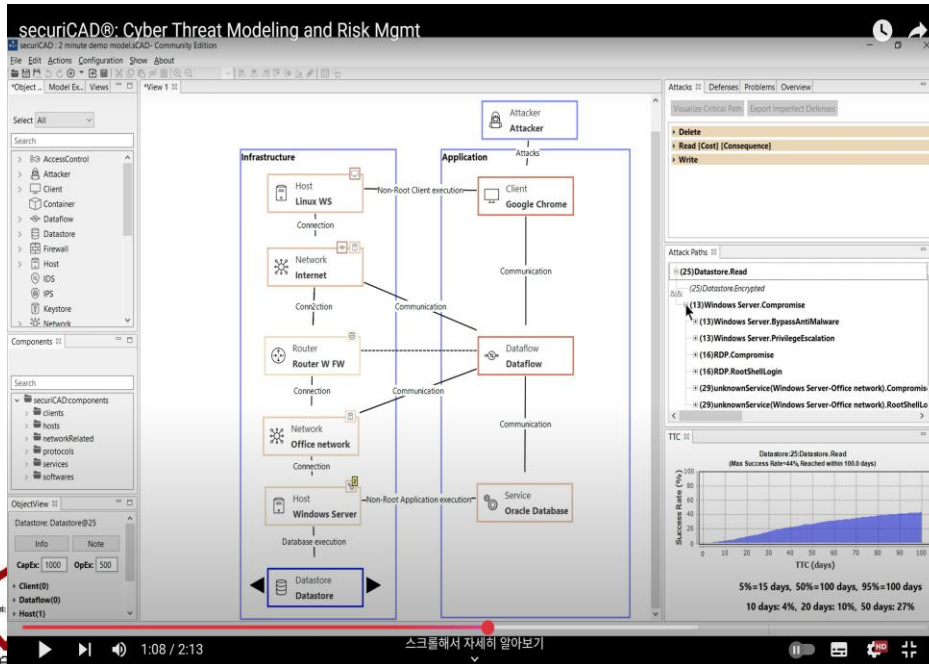
- 각 노드별 확률 밀도 함수에 따라 난수 생성
 - 공격 단계일 경우 local TTC
 - 방어 기법일 경우 성공 여부
 - ...
- entry point들로부터 target까지 local TTC들의 합, 즉, global TTC를 가장 적게 하는 경로 도출
- 위 과정을 여러 번 수행하여, 가장 많이 방문한 노드 및 경로 기록
 - 가장 위험한 경로 (entry point들로부터 가장 빠른 시간에 target에 도달할 수 있을 확률이 가장 높은 경로)

6-2. 공격 시뮬레이션 수행 - 도구

■ securiCAD^[14]

■ MAL기반 위협 모델링 및 보안 평가 도구

- MAL로 명세된 DSL을 활용하여 IT 인프라 모델링
- 시스템 모델에서 발생할 수 있는 다양한 공격 시나리오에 대한 확률적 공격 시뮬레이션 자동 수행 (몬테카를로 방법)
- 시뮬레이션 결과 레포트화 및 **Critical Path** 시각화
- 특정 시나리오나 설정에 대해 시뮬레이션을 조정하고, 특정 보안 목표를 달성하기 위한 방안을 찾을 수 있음



■ securiCAD

■ securiCAD의 역사

■ 2014년 Foreseeti 설립

- 스웨덴 KTH Royal Institute of Technology에서 설립된 스피노프 회사



■ 2015년 securiCAD가 Foreseeti에 의해 개발, 논문으로 발표됨[15]

- SecuriCAD by Foreseeti: A CAD Tool for Enterprise Cyber Security Management

■ 2021년 securiCAD Vanguard가 AWS Marketplace에 제품으로 등록

- 현재는 제공되지 않음



■ 2022년 Google이 Foreseeti를 인수하며 현재는 Google cloud 환경에 사용됨[16]

- 공개적 접근성 제한

Our attack path simulation technology comes from Foreseeti, a Swedish risk analytics company acquired by Google in 2022. Foreseeti was founded by university researchers and scientists who were inspired by large-scale simulations in other industries, including automobile crash tests and the structural analysis of bridges, and sought to apply their research and techniques to improve cybersecurity.

- **malsim**_[17]

- **mal-lang** **github**에서 제공하는 기본 공격 시뮬레이션 도구
- **TTC** 확률 밀도를 고려하지 않음

- **sim.py**

- **securiCAD의 작동 원리(알고리즘 및 시각화)를 최대한 모방한 공격 시뮬레이션 도구**
 - 몬테 카를로 기반 시뮬레이션
 - **Critical Paths** 시각화 지원
- 기존 **malc**, **mal-gui**, **maltoolbox**를 이용하여 생성한 공격 그래프 (**attackgraph.yml**) 활용
- 다양한 **MAL** 기반 **DSL** 모델의 공격 시나리오에 적용
 - 각 **DSL**을 제안한 논문에서 제시하는 사례와 유사한 결과 도출
 - 시스템 모델 – 공격 시나리오(**entry points**와 **target**) – **critical paths** 및 **global TTC**

- **sim.py**
 - 알고리즘

Algorithm: BuildValidPaths

Input: G, EP, T

Output: S = (nodes, edges)

nodes $\leftarrow$ DFSAllPaths(EP, T, G)

nodes $\leftarrow$ FilterByExistConditions(nodes, G)

SetDefenseInfo(nodes, G)

edges $\leftarrow$ BuildEdges(nodes, G)

return (nodes, edges)

Algorithm: Simulate

Input: S, EP, T, iterations

Output: paths, ttcs, edgeCounts

for i $\leftarrow$ 1 to iterations do

 path, ttc $\leftarrow$ RunIteration(S, EP, T)

 if ttc $\neq \infty$ then

 Record(path, ttc, edgeCounts)

return results

- **sim.py**
 - 알고리즘

Algorithm: RunIteration

Input: S, EP, T

Output: path, ttc

// Sample TTC and apply defenses

for each node $\in$ S.nodes do

 node.ttc $\leftarrow$ SampleTTC(node.distribution)

 if node.defended and Random() < node.defenseProb then

 node.ttc $\leftarrow \infty$

// Remove blocked nodes

activeNodes $\leftarrow$ {node : node.ttc $\neq \infty$ }

if T $\notin$ activeNodes then return $\emptyset, \infty$

return EventSimulation(activeNodes, S.edges, EP, T)

- **sim.py**
 - 알고리즘

Algorithm: EventSimulation

Input: nodes, edges, EP, T

Output: path, ttc

```
events ← PriorityQueue()
completed ← {}
```

```
// Initialize entry points
```

```
for each ep  $\in$  EP do
```

```
    events.push(0, ep)
```

```
    completed[ep] ← 0
```

```
while events not empty do
```

```
    time, node ← events.pop()
```

```
if node = T then
```

```
    return BuildPath(), time
```

```
// Process children
```

```
for each (node, child)  $\in$  edges do
```

```
    if CanStart(child, completed) then
```

```
        startTime ← GetStartTime(child, completed)
```

```
        events.push(startTime + child.ttc, child)
```

```
        completed[child] ← startTime + child.ttc
```

```
return  $\emptyset$ ,  $\infty$ 
```



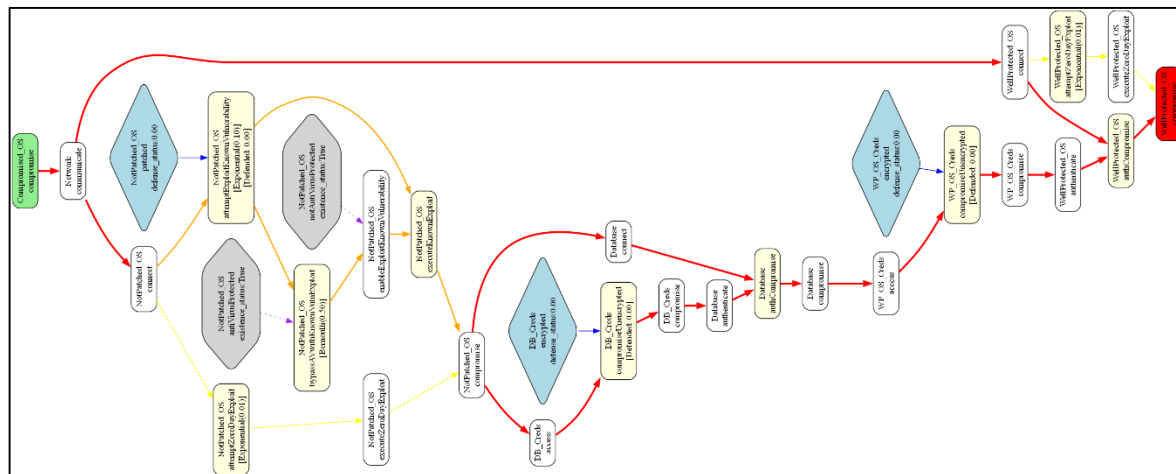
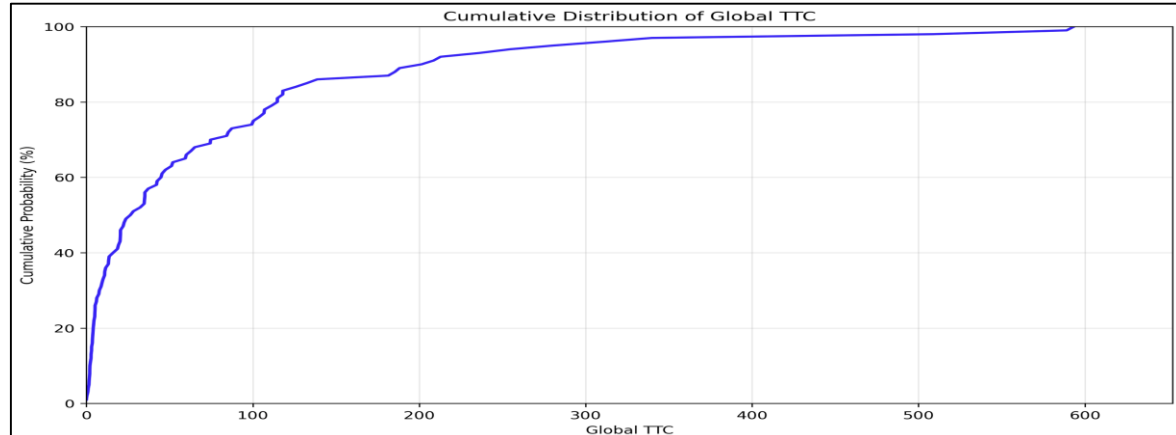
6-2. 공격 시뮬레이션 수행 - 도구

▪ sim.py

▪ cmd에 명령어 입력

> python sim.py [attackgraph.yml] [entry_point_count] [entry_point1] ... [target]
[iterations] [hidden 0 or 1]

```
Starting simulation:
YAML file: .\TestLang_models\attackgraph.yml
Entry points: ['Compromised_OS:compromise']
Target: WellProtected_OS:compromise
Iterations: 1000
Hide hidden nodes: False
Loading .\TestLang_models\attackgraph.yml...
Loaded: 85 attack steps, 6 defenses, 8 exist nodes
Entry points: [84]
Target: 45
Starting simulation with 1000 iterations...
Hide hidden nodes: False
=== FINDING ENTRY-TO-TARGET PATHS ===
Finding entry-to-target paths...
Found 37 nodes on entry-to-target paths
Applying exist conditions...
Removed 0 nodes due to exist conditions
Setting defense information...
Defended nodes: 4
Built 50 path edges (target has no outgoing edges)
Final paths: 37 nodes, 50 edges
Generating allpaths.png (showing all nodes)...
Generated allpaths.png
Progress: 100/1000 (10.0%)
Progress: 200/1000 (20.0%)
Progress: 300/1000 (30.0%)
Progress: 400/1000 (40.0%)
Progress: 500/1000 (50.0%)
Progress: 600/1000 (60.0%)
Progress: 700/1000 (70.0%)
Progress: 800/1000 (80.0%)
Progress: 900/1000 (90.0%)
Progress: 1000/1000 (100.0%)
Success: 1000/1000 (100.0%)
Generating critical_paths.png (showing all nodes)...
Generated critical_paths.png
Generating global_ttc.png...
Generated global_ttc.png
Simulation completed successfully!
```

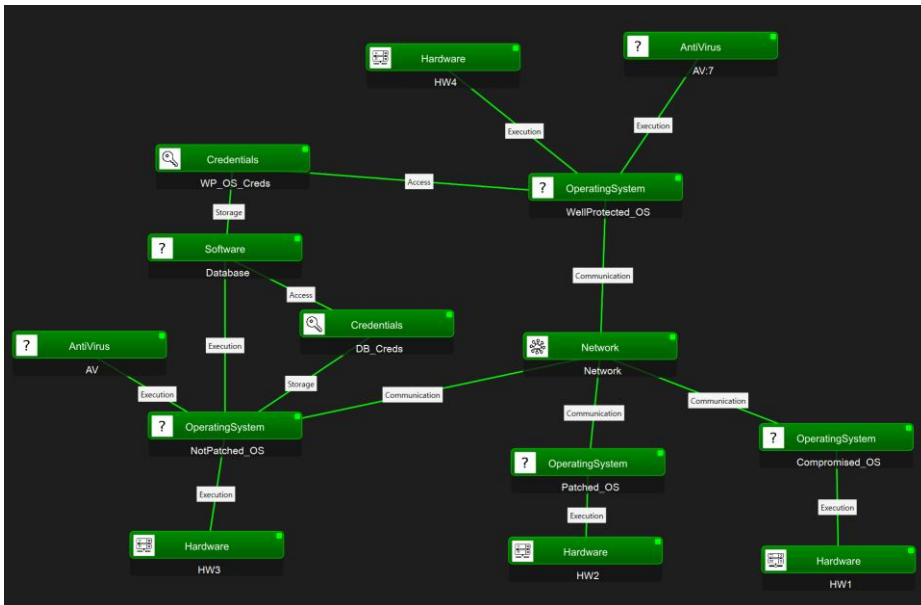


6-3. 공격 시뮬레이션 수행 - 적용

▪ sim.py

▪ 실제 securiCAD 사례 적용

- The meta attack language - a formal description 예시[3]
- 시스템 모델링



mal-gui로 모델링

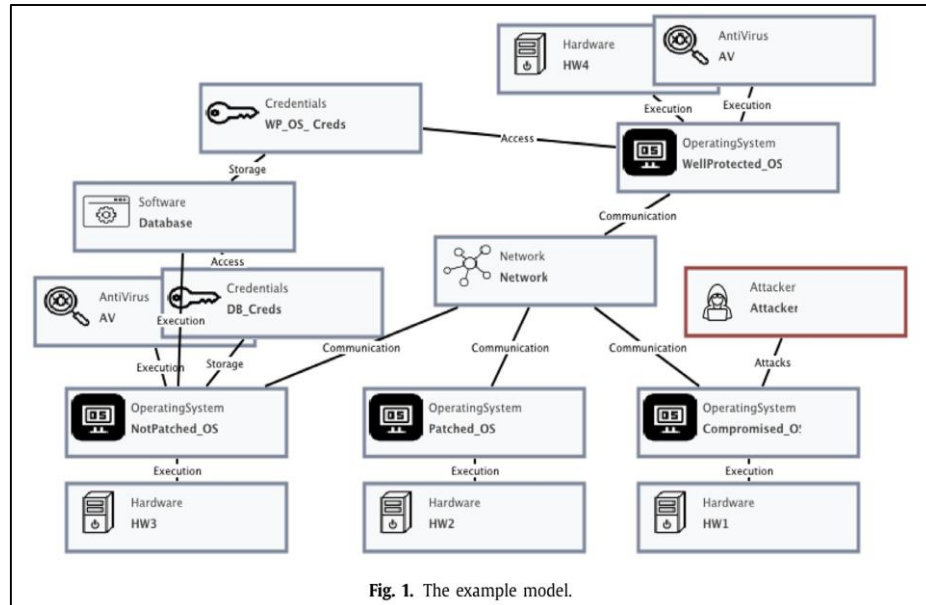


Fig. 1. The example model.

securiCAD로 모델링

6-3. 공격 시뮬레이션 수행 - 적용

▪ sim.py

▪ 실제 securiCAD 사례 적용

- The meta attack language - a formal description 예시
- 공격 시뮬레이션 수행

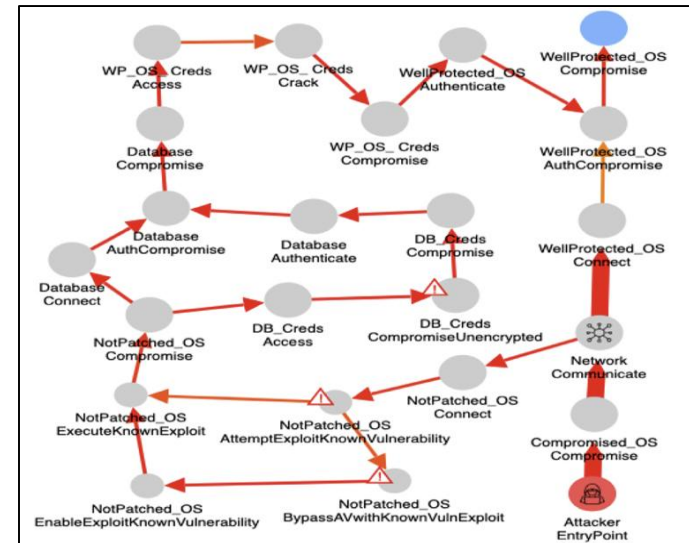
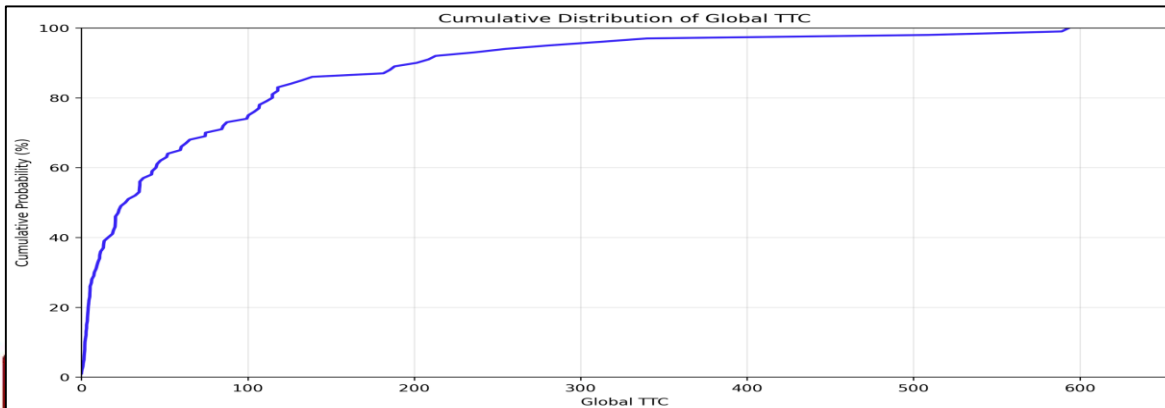
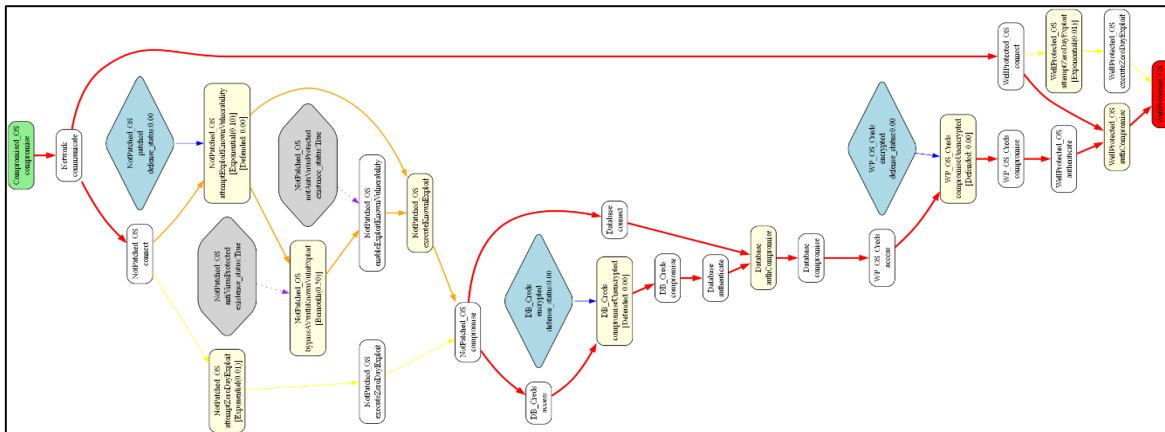


Fig. 2. The shortest attack vector to the target operating system.

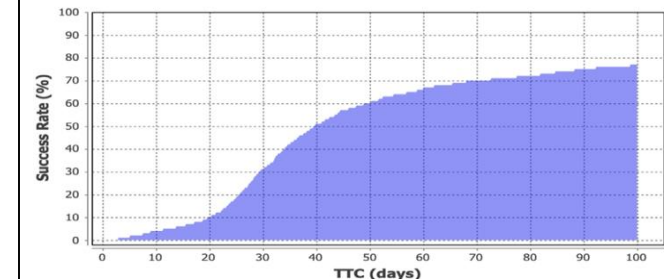
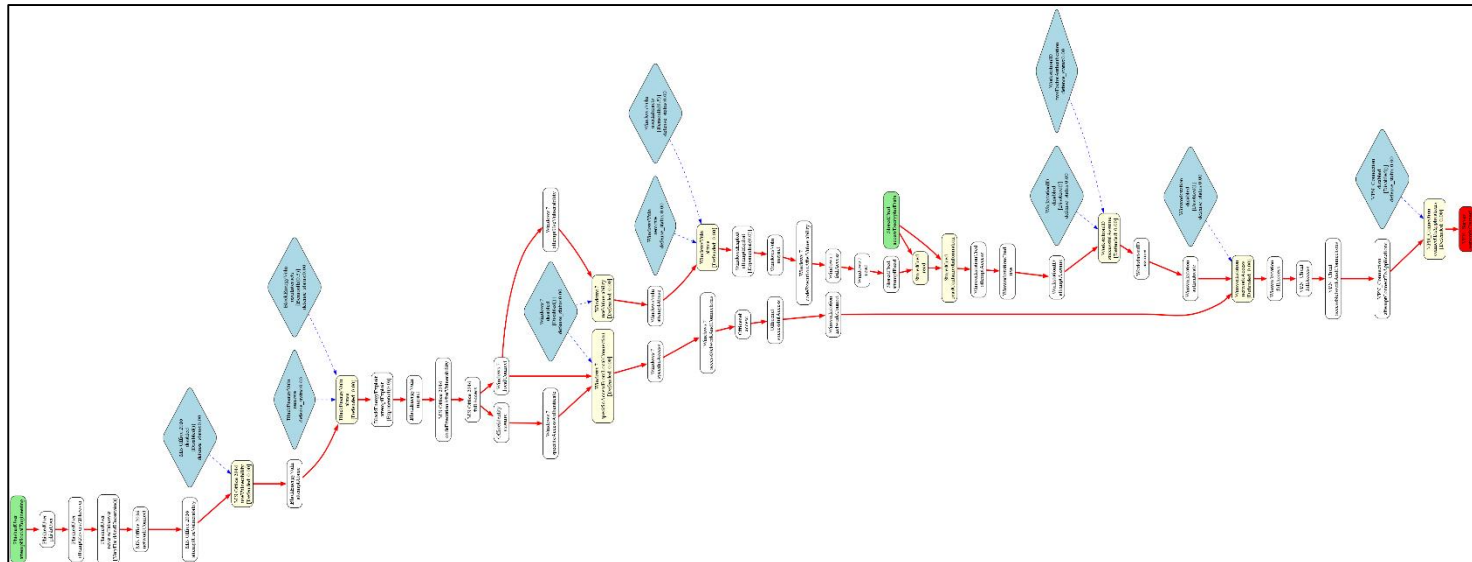
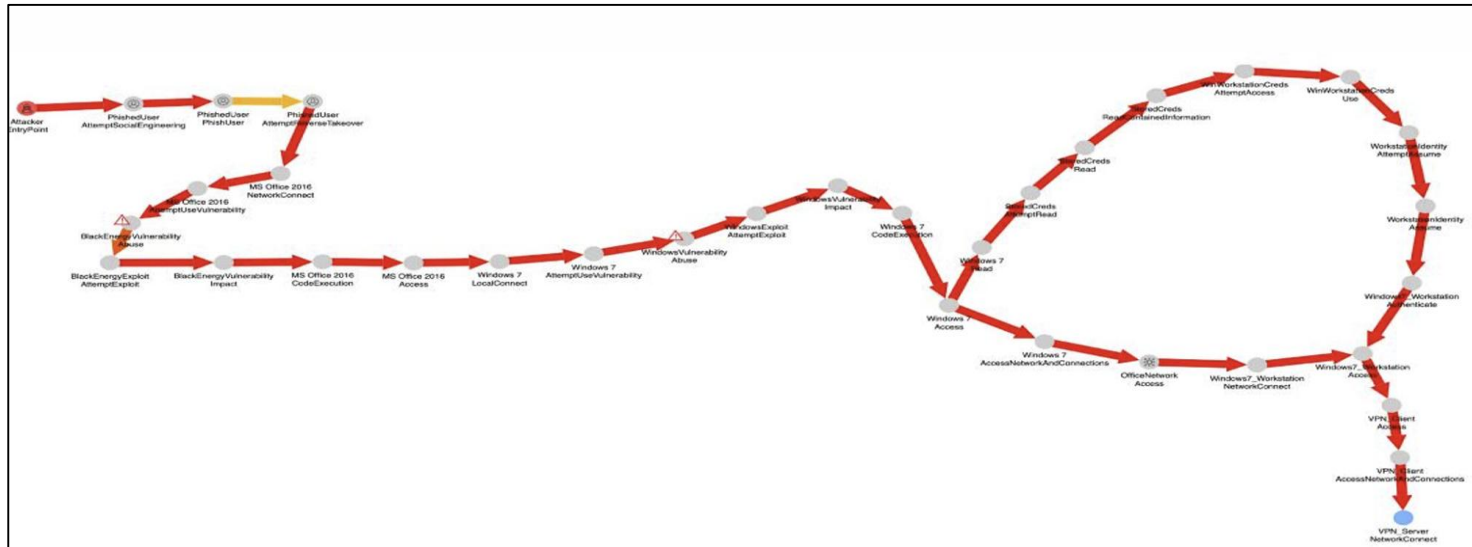


Fig. 3. The cumulative distribution function of the global TTC for the target attack step *Compromise* on the asset *WellProtected_OS*.

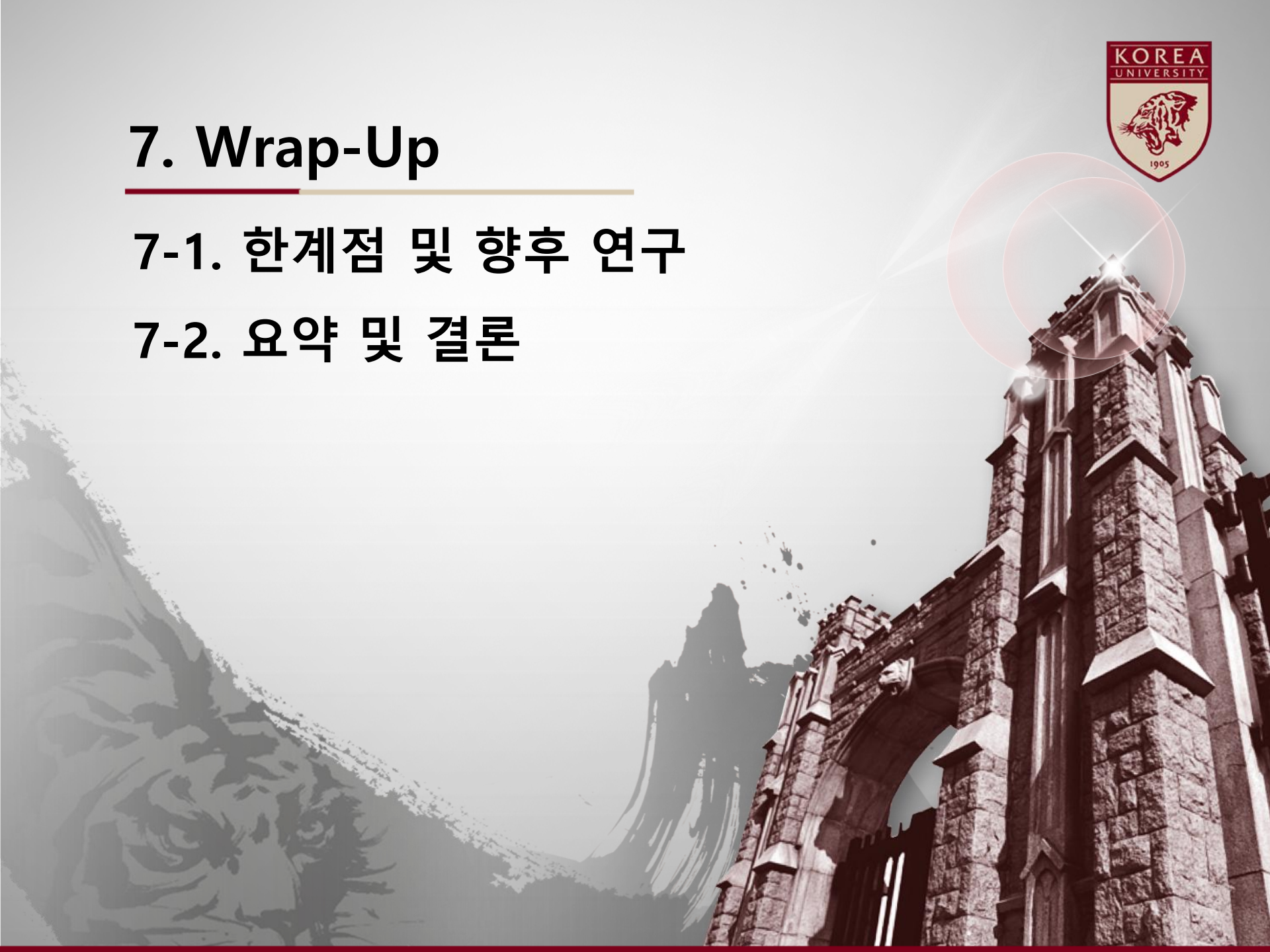
- **coreLang0.1.1 – 우크라이나 전력망 공격 사례**^[18]



7. Wrap-Up

7-1. 한계점 및 향후 연구

7-2. 요약 및 결론



- 공격 시뮬레이션 도구 **sim.py**의 한계
 - 시뮬레이션 알고리즘 및 시각화 방식 개선 필요
 - MAL에서 정의한 모든 **syntax** 활용 불가 (개선 예정)
- 향후 연구
 - 공격 시뮬레이션 결과 **Critical Path**는 확률적으로 “가장 위험한” 경로
 - 공격자가 공격 단계를 “선택” 하는 개념 반영 -> 더욱 현실적인 시뮬레이션 가능
 - 전 분포/후 분포를 고려한 **TTC** 확률 구조 모델링
 - 공격 단계들을 성공시키는데 있어 걸리는 시간이 독립적인 것이 아니라 상호 종속적
 - Ex) User:guessPassword1 성공 시 User:guessPassword2가 성공할 시간 단축
 - 시스템 운용 환경을 유동적으로 반영
 - 공격자의 **attack profile** 반영
 - 공격자의 특성에 따라 공격 단계 및 **TTC** 확률 구조 변경
 - 다수의 공격자, 다수의 **target** 조합에도 적용 가능한 시뮬레이션
 - 시스템의 **defense profile** 반영
 - 어떠한 보안 조치가 각 **defense step**의 방어 확률에 어떻게 영향을 미치는가를 반영

■ MAL 기반 위협 모델링 workflow 요약

- 1) MAL 기반 DSL 생성 및 컴파일(malc 등)/ 기존 DSL 파일 다운로드
- 2) DSL 규칙에 맞게 시스템 모델링(mal-gui 등)
- 3) 공격 그래프 생성(view_attack_graph.py 등)
- 4) 공격 시뮬레이션 시나리오 설정
 - 방어 기법 설정
 - entry point(s)
 - Target
- 5) 공격 시뮬레이션 수행 (sim.py 등)
- 6) 도출된 공격 시나리오(global TTC 분포 및 Critical Paths) 분석
 - 자산에 대한 방어 기법 추가/강화, 시스템 구성 변경 등
- 7) 2) ~ 6) 반복

■ 결론

- “왜 MAL을 사용하는가”를 구체적으로 알아봄
- 제한적인 실습 환경에서 MAL 기반 공격 시뮬레이션 수행



8. 부록

8-1. Prerequisites

8-2. 도구 설치 방법

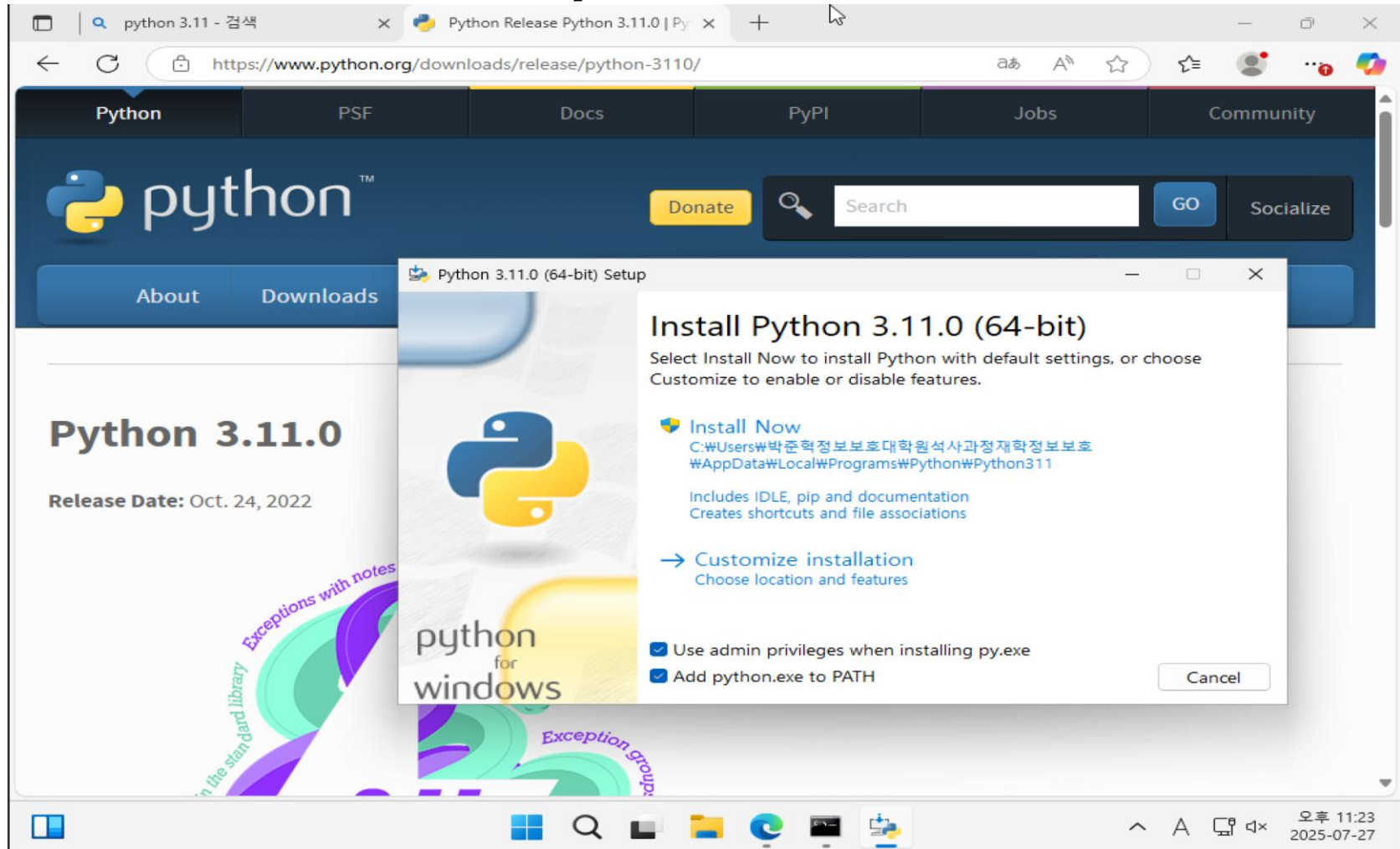
8-3. 참고 문헌



8-1. 부록 – Prerequisites

▪ Python 3.11 설치

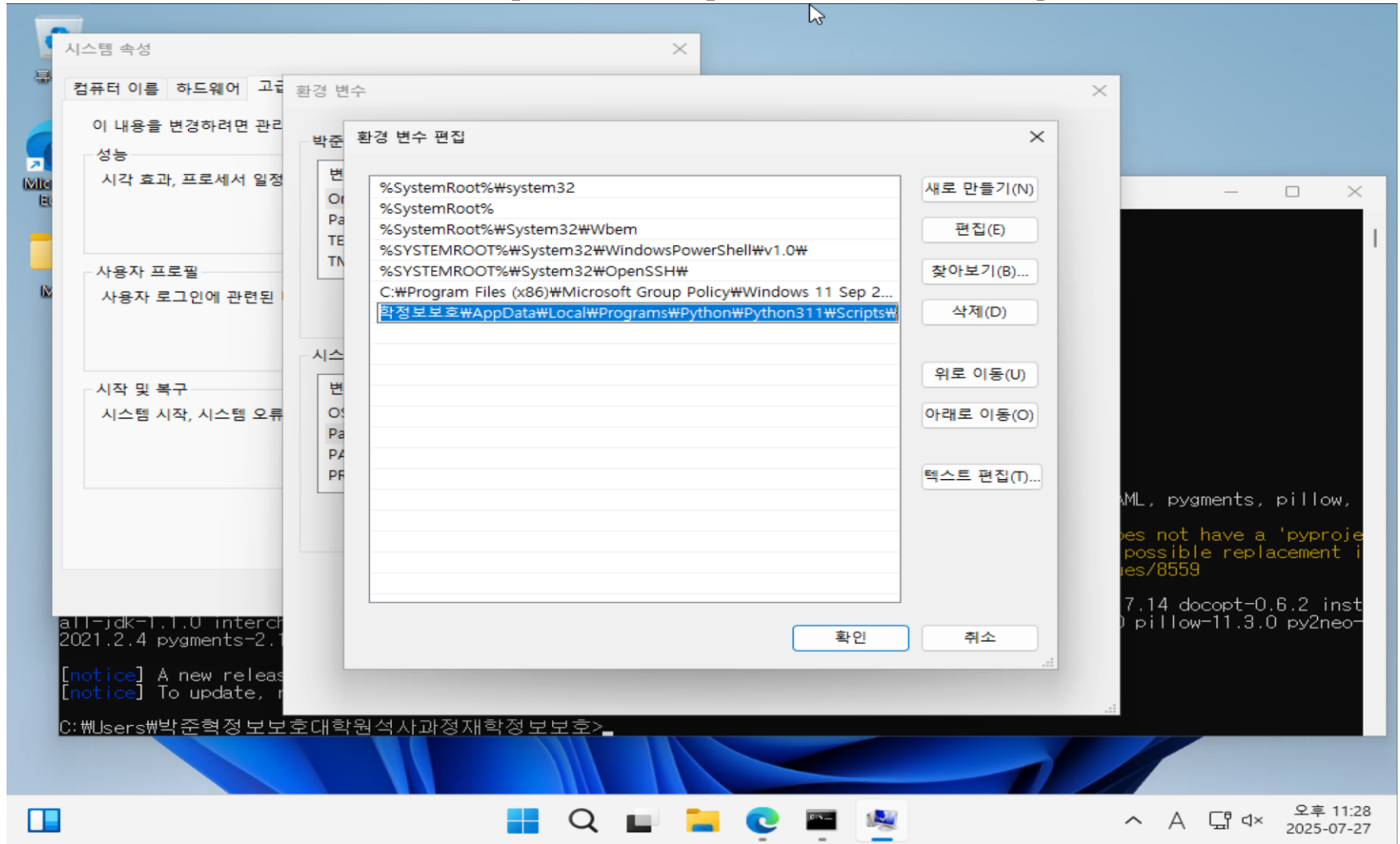
▪ 3.11 이상 버전의 경우 Pyside 라이브러리와 충돌



8-1. 부록 – Prerequisites

▪ Python 3.11 설치

- 새로 만들기 -> ...□Python□Python311□Scripts 환경 변수 추가



■ Graphviz 설치

- <https://graphviz.org/download/> 접속
- 플랫폼에 맞는 EXE installer 설치

The screenshot shows the Graphviz website's download page. The browser address bar displays 'graphviz.org/download/'. The page has a dark blue header with the 'Graphviz' logo and navigation links: 'Download', 'Documentation', 'Gallery', 'Forum', 'GitLab', and 'Search this'. A left sidebar contains a search bar and a list of links: 'About', 'Download', 'Source Code', 'Documentation', 'DOT Language', 'Command Line', 'Layout Engines', 'Output Formats', 'Attributes', 'Attribute Types', and 'Graph Attributes'. The main content area lists operating systems: 'Fedora project*', 'Rocky Linux', 'Redhat Enterprise Linux, or CentOS*'. Below this, a code block shows the command: `sudo dnf install graphviz`. The 'Windows' section lists 'Stable Windows install packages, built with Microsoft Visual Studio 16 2019:'. Under 'graphviz-13.1.1', there are four links: 'graphviz-13.1.1 (64-bit) ZIP archive [sha256] (contains all tools and libraries)', 'graphviz-13.1.1 (64-bit) EXE installer [sha256]', 'graphviz-13.1.1 (32-bit) EXE installer [sha256]', and 'graphviz-13.1.1 (32-bit) ZIP archive [sha256] (contains all tools and libraries)'. The 'graphviz-13.1.1 (64-bit) EXE installer [sha256]' link is highlighted with a red rectangle.

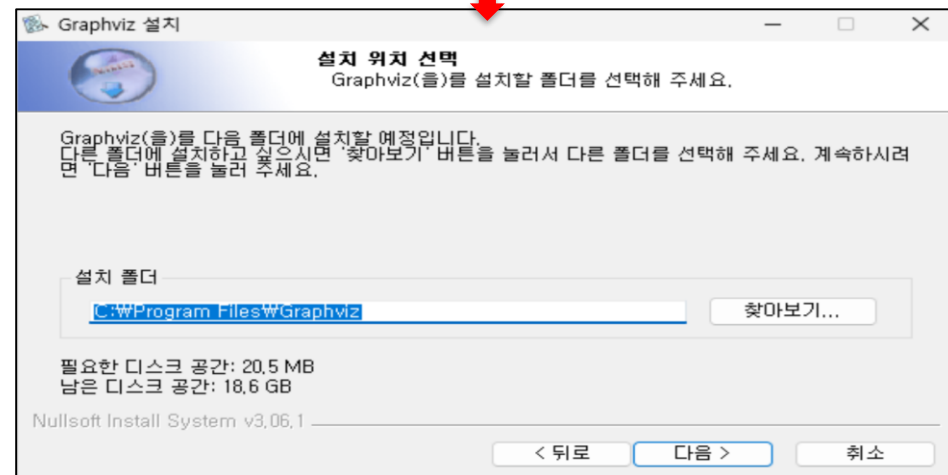
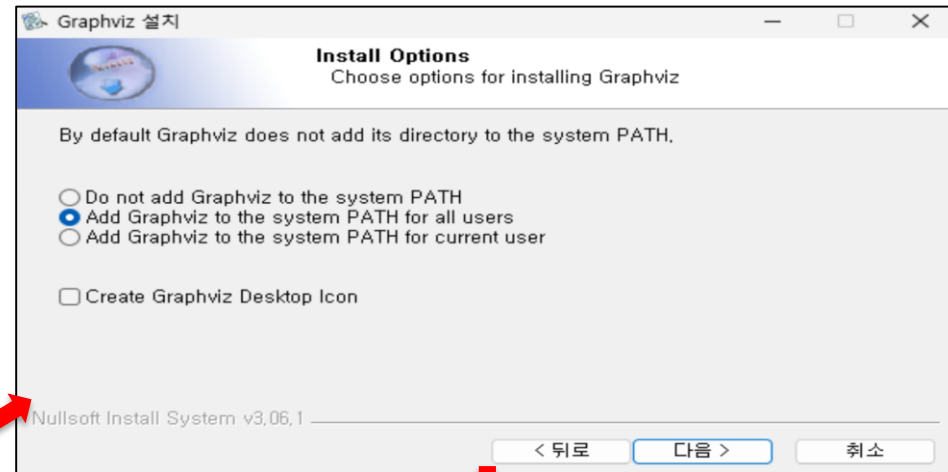
8-1. 부록 – Prerequisites

■ Graphviz 설치

■ installer 실행 및 환경변수 자동 편집

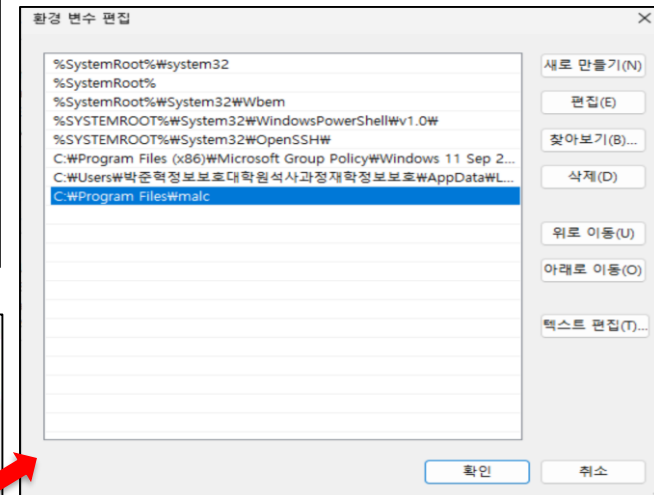
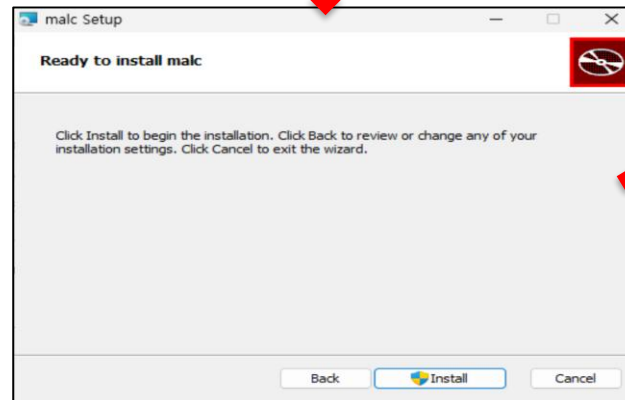
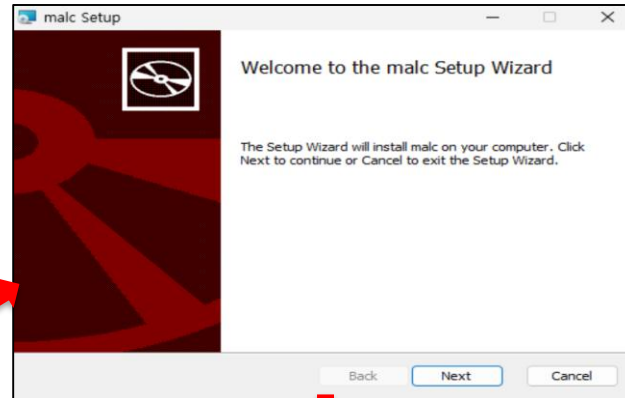
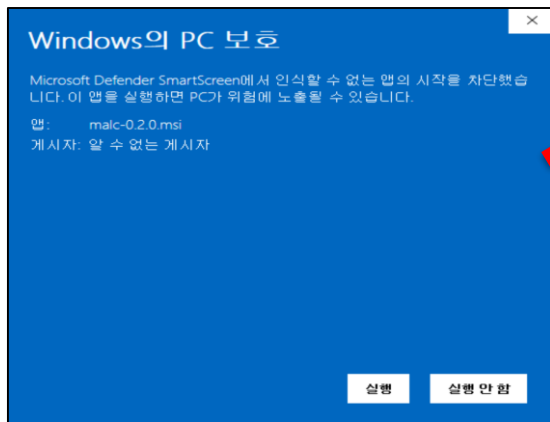
■ 파이썬 패키지 설치

- > pip install pyyaml
- > pip install graphviz
- > pip install matplotlib



■ malc 설치

- <https://github.com/mal-lang/malc/releases/> 접속
- malc-0.2.0.msi 다운로드 및 실행 -> malc Setup에서 연속적으로 Next 선택 -> malc Setup install 완료 -> 시스템 환경 변수에 malc 경로 추가



8-2. 부록 – 도구 설치

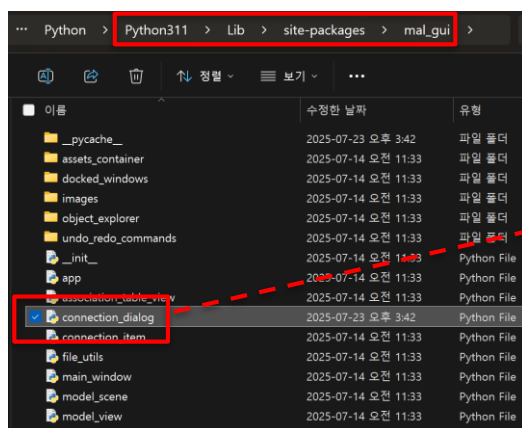
■ mal-gui 설치

■ cmd에 명령어 입력

> pip install mal-gui

■ mal-gui 버그 수정

- 자산 간 association 선택 시 자산 type 키워드 검색 오류로 상속 관계를 가진 association이 filtering 되지 않아 정상적인 연결이 불가능한 버그
- connection_dialog.py를 수정 버전으로 대체



```
possible_assocs = self.start_asset.lg_asset.associations
for fieldname, association in possible_assocs.items():
    field = association.get_field(fieldname)

    # If assoc ends with end_assets type, give that assoc
    # as option in list widget
    if field.asset == self.end_asset.lg_asset:
        assoc_list_item = QListWidgetItem(self.start_asset.name +
        assoc_list_item.setData(
            Qt.UserRole,
            {
                'from': self.start_asset,
                'to': self.end_asset,
                'fieldname': fieldname
            }
        )
        self.association_list_widget.addItem(assoc_list_item)

self.layout.addWidget(self.association_list_widget)
```

```
possible_assocs = self.start_asset.lg_asset.associations
for fieldname, association in possible_assocs.items():
    # start_asset과 end_asset이 association의 양쪽 field에 속하는 경우에만
    start_matches_left = self.start_asset.lg_asset.is_subasset_of(association.left_field.asset)
    start_matches_right = self.start_asset.lg_asset.is_subasset_of(association.right_field.asset)
    end_matches_left = self.end_asset.lg_asset.is_subasset_of(association.left_field.asset)
    end_matches_right = self.end_asset.lg_asset.is_subasset_of(association.right_field.asset)

    # start_asset이 왼쪽에, end_asset이 다른 쪽에 매칭되어야 함
    valid_connection = (
        (start_matches_left and end_matches_right) or
        (start_matches_right and end_matches_left)
    )

    if valid_connection:
        assoc_list_item = QListWidgetItem(self.start_asset.name + "." + fieldname + " = " + self.end_asset.name)
        assoc_list_item.setData(
            Qt.UserRole,
            {
                'from': self.start_asset,
                'to': self.end_asset,
                'fieldname': fieldname
            }
        )
        self.association_list_widget.addItem(assoc_list_item)

self.layout.addWidget(self.association_list_widget)
```

■ maltoolbox 설치

■ cmd에 명령어 입력

> pip install mal-toolbox



- [1] Ji, L., Ma, J., & Shan, Z. (2012) Research on Model Checking Technology of UML
- [2] Jürjens, J. (2002) UMLsec: Extending UML for Secure Systems Development
- [3] Wideł, W., Hacks, S., Ekstedt, M., Johnson, P., & Lagerström, R. (2023) The Meta Attack Language - a Formal Description
- [4] <https://github.com/mal-lang/mal-documentation/wiki/MAL-Syntax>
- [5] <https://github.com/mal-lang/coreLang>
- [6] <https://github.com/mal-lang/malcompiler/wiki/Supported-distribution-functions>
- [7] <https://github.com/mal-lang/malc>
- [8] https://github.com/mal-lang/mal-toolbox-tutorial/blob/main/guides/create_model.md
- [9] <https://github.com/mal-lang/mal-gui>
- [10] <https://github.com/mal-lang/mal-toolbox>
- [11] <https://github.com/mal-lang/mal-toolbox-tutorial/blob/main/guides/visualize.md>
- [12] Katsikeas, S., Buhaiu, A., Ekstedt, M., Afzal, Z., Hacks, S., & Mukherjee, P. (2024) Development and validation of coreLang: A threat modeling language for the ICT domain

- [13] Gylling, A., Ekstedt, M., Afzal, Z., & Eliasson, P. (2021) Mapping Cyber Threat Intelligence to Probabilistic Attack Graphs
- [14] <https://www.youtube.com/watch?v=B88J1LOhMtc>
- [15] Ekstedt, M., Johnson, P., Lagerström, R., Gorton, D., Nydrén, J., & Shahzad, K (2015) securiCAD by foresee: A CAD tool for enterprise cyber security management
- [16] <https://cloud.google.com/blog/products/identity-security/adding-attack-path-simulation-technology-to-security-command-center?hl=en>
- [17] <https://github.com/mal-lang/mal-simulator>
- [18] Katsikeas, S. (2020) An Attack Simulation Language for the IT Domain

“ 대한민국을 이끌어온 자랑스럽고 위대한 역사

눈부신 교재의 새로운 백년도 지켜가겠습니다.”

Global KU - Frontier Spirit

KU is committed to becoming a leading world-class university

